A Study on Overcommitment in Cloud Providers

Jiang Zhi

Thesis presented to the
Institute of Mathematics and Statistics
of the University of São Paulo
in partial fulfillment
of the requirements
for the degree of
Master of Science

Program: Computer Science
Advisor: Prof. Dr. Alfredo Goldman

This study was financed in part by the Coordenação de Aperfeiçoamento de Pessoal de Nível Superior – Brasil (CAPES) – Finance Code 001

São Paulo
February, 2025

A Study on Overcommitment in Cloud Providers

Jiang Zhi

This version of the thesis includes the corrections and modifications suggested by the Examining Committee during the defense of the original version of the work, which took place on February 6, 2025.

A copy of the original version is available at the Institute of Mathematics and Statistics of the University of São Paulo.

Examining Committee:

Prof. Dr. Alfredo Goldman vel Lejbman – IME-USP
Profª. Drª. Sarita Mazzini Bruschi – ICMC-USP
Profª. Drª. Priscila América Solis Mendez – UnB

The content of this work is published under the CC BY 4.0 license
(Creative Commons Attribution 4.0 International License)

Ficha catalográfica elaborada com dados inseridos pelo(a) autor(a)
Biblioteca Carlos Benjamin de Lyra
Instituto de Matemática e Estatística
Universidade de São Paulo

Zhi, Jiang
Um estudo de sobrealocação em provedores de nuvem / Jiang Zhi; orientador, Alfredo Goldman vel Lejbman. - São Paulo, 2025.
82 p.: il.

Dissertação (Mestrado) - Programa de Pós-Graduação em Ciência da Computação / Instituto de Matemática e Estatística / Universidade de São Paulo.
Bibliografia
Versão corrigida

1. COMPUTAÇÃO EM NUVEM. 2. ALOCAÇÃO DE RECURSOS. 3. SOBREALOCAÇÃO. I. Goldman vel Lejbman, Alfredo. II. Título.

Bibliotecárias do Serviço de Informação e Biblioteca
Carlos Benjamin de Lyra do IME-USP, responsáveis pela
estrutura de catalogação da publicação de acordo com a AACR2:
Maria Lúcia Ribeiro CRB-8/2766; Stela do Nascimento Madruga CRB 8/7534.

When everything seems to be going against you,
remember that the airplane takes off against the wind, not with it.
— Henry Ford

Acknowledgements

I am extremely grateful to my supervisor Dr. Alfredo Goldman for his guidance through the writing of the thesis. During this period, my family has provided support and assistance, for which I am very indebted. I also want to thank my friends and colleagues for the adventures and memorable experiences shared during university.

I would like to express my sincere gratitude to the Coordenação de Aperfeiçoamento de Pessoal de Nível Superior – Brasil (CAPES) for providing the scholarship that supported this research. I acknowledge the use of generative AI and grammar correction tools, which have greatly contributed to improving the clarity and grammar of the thesis. Without the tools, the thesis would not be as polished as it is now.

Resumo

Jiang Zhi. **Um estudo de sobrealocação em provedores de nuvem**. Dissertação (Mestrado). Instituto de Matemática e Estatística, Universidade de São Paulo, São Paulo, 2025.

A computação em nuvem se tornou uma tecnologia essencial no século XXI, amplamente reconhecida pela escalabilidade, pelo custo-benefício e pela flexibilidade. Apesar de suas vantagens, as ineficiências no uso de recursos continuam sendo um desafio significativo para os provedores de serviços em nuvem. Por exemplo, estudos sobre o Google Borg, um sistema de gerenciamento de clusters, mostraram que apenas cerca de 20% da capacidade computacional e 40% da memória são utilizados, deixando uma quantidade substancial de recursos subutilizados. Essas ineficiências são geralmente causadas por usuários que superestimam suas necessidades de recursos e pelas complexidades de gerenciar cargas de trabalho em ambientes imprevisíveis.

Este estudo foca nas políticas de sobrealocação como uma solução para melhorar a utilização de recursos. A sobrealocação envolve a alocação de mais recursos virtuais do que o hardware físico pode suportar, aproveitando de forma eficaz a capacidade subutilizada. Gerenciando com cuidado a qualidade de serviço (QoS), essa técnica pode aumentar a eficiência sem comprometer o desempenho. A pesquisa analisa estratégias do estado da arte em sobrealocação, introduz uma ferramenta de coleta de dados para obter informações do traço de cargas de trabalho de provedores de nuvem e desenvolve um simulador para avaliar essas políticas em diversos cenários de carga de trabalho. O objetivo da dissertação é desenvolver ferramentas para estudar a sobrealocação, visando minimizar o desperdício de recursos e melhorar o desempenho de ambientes de computação em nuvem.

Palavras-chave: Computação em nuvem. Alocação de recursos. Escalonamento. Sobrealocação.

Abstract

Jiang Zhi. **A Study on Overcommitment in Cloud Providers**. Thesis (Master's). Institute of Mathematics and Statistics, University of São Paulo, São Paulo, 2025.

Cloud computing has become a key technology of the 21st century, widely recognized for its scalability, cost-effectiveness, and flexibility. Despite its advantages, resource inefficiencies remain a significant challenge for cloud service providers. For example, studies on Google Borg, a cluster management system, showed that only about 20% of computational capacity and 40% of memory are utilized, leaving substantial resources unused. These inefficiencies are often caused by users overestimating their resource needs and the complexities of managing workloads in unpredictable environments.

This study focuses on overcommitment policies as a solution to improve resource utilization. Overcommitment involves allocating more virtual resources than the physical hardware can support, effectively leveraging underutilized capacity. By carefully managing quality of service (QoS), this technique can boost efficiency while maintaining performance. The research examines state-of-the-art overcommitment strategies, introduces a data collection tool to gather workload trace information from cloud service providers, and develops a simulator to evaluate these policies under diverse workloads. The goal of this thesis is to develop tools for studying overcommitment, aiming to minimize resource waste and enhance the performance of cloud computing environments.

Keywords: Cloud computing. Resource allocation. Scheduling. Overcommitment.

List of Abbreviations

VM	Virtual Machine
PM	Physical Machine
QoS	Quality of Service
SLA	Service Level Agreement
I/O	Input and Output
OOM	Out of Memory
CSP	Cloud Service Provider
AWS	Amazon Web Services
GCP	Google Cloud Platform

List of Figures

2.1 Example of CPU overcommitment 8

3.1 Literature review 16
3.2 Difference between VBP and MBP 18

4.1 Methodology design 23

5.1 Cloud simulator diagram 27
5.2 Evaluation method 31
5.3 Simulator — CPU usage 33

7.1 Comparison between CPU usage and latency 47
7.2 Alibaba - CPU 49
7.3 Alibaba - Memory 49

List of Tables

2.1 Dynamic overcommitment 13

3.1 Current work comparison 20

4.1 Artifact for experiments 22

5.1 Overload handling method 30

6.1 Dataset comparison - Container 39
6.2 Dataset comparison - Extra 40

7.1 Hardware specification 45
7.2 TailBench workloads 46
7.3 Result from the overcommitment experiments 51

Contents

1 Introduction **1**

1.1 Motivation 2
1.2 Objective 3
1.3 Contribution 3
1.4 Thesis organization 3

2 Overcommitment **5**

2.1 Cloud computing 5
2.2 Resources types 6
2.3 Overcommitment 7
2.4 Problem formulation 9
2.5 Resource overload 10
2.6 Slack and automation 10
2.7 QoS reduction 11
2.8 Overcommitment policies 11
2.8.1 Machine-centric 12
2.8.2 Container-centric 12
2.9 Summary 13

3 Literature review **15**

3.1 Cloud service provider 17
3.2 Resource optimization 17
3.3 Overcommitment 18
3.4 Vector bin-packing 18
3.5 Overload mechanism 19
3.6 Comparison with existing simulator 19
3.7 Summary 20

4 Methodology **21**
4.1 Design 22
4.2 Summary 24

5 Cloud simulator **25**
5.1 Comparison of different cloud simulators 25
5.2 Clovers — cloud overcommitment simulator 26
5.3 Components of the simulator 26
5.4 Engine 27
5.5 Overcommitment policies 28
5.6 Scheduler 28
5.6.1 Scheduling algorithms 28
5.7 Overload handler 29
5.8 Evaluation metrics 30
5.9 Implementation 31
5.9.1 Simulator implementation 32
5.9.2 Calculating the percentile of resources 33
5.10 Summary 35

6 Data collection **37**
6.1 Trace data source 37
6.1.1 Google Borg 37
6.1.2 Azure 38
6.1.3 Alibaba Cloud 38
6.1.4 Synthesized workload 39
6.1.5 Comparison 39
6.2 Container classification 40
6.3 Implementation 41
6.3.1 Tools 41
6.3.2 Data extraction process 41
6.3.3 Alibaba Fuxi/Sigma 42
6.3.4 Microsoft Azure 42
6.3.5 Google Borg 42
6.4 Summary 44

7 Experiments **45**
7.1 CPU usage effects on latency 45
7.2 Evaluation of resource underutilization 48
7.3 Comparison between overcommitment policies 50
7.3.1 Machine-centric overcommitment 50
7.3.2 Container-centric overcommitment 50
7.4 Summary 51

8 Conclusion **53**
8.1 Future direction 54
8.1.1 Comparison of resource underutilization among other CSPs . . . 54
8.1.2 Comparison of different time windows in resource history 55
8.1.3 Comparison among scheduling algorithms 55
8.1.4 Comparison among overload handling 55
8.1.5 Automation of overcommitment policies 55
8.1.6 Clustering-based overcommitment policies 55
8.1.7 Machine learning in overcommitment policies 56

References **57**

Chapter 1

Introduction

Cloud computing is a rapidly evolving technology that has transformed the way organizations and individuals utilize computational resources. This technology enables users to process, store, and access data at any time, as long as an internet connection is available.

A foundational concept that enables cloud computing is virtualization, which involves the creation of a software abstraction over computer hardware to share the processing, storage, and network devices of a physical machine among multiple users. At the beginning, virtualization was implemented solely through virtual machines (VMs). VMs are software-based emulations of a physical computer that allow multiple operating systems to run simultaneously on a single physical machine. Each VM operates with dedicated resources managed by a hypervisor, such as Xen (Xen Project, n.d.), KVM (Linux Kernel Virtual Machine, n.d.), or ESXi (VMware, n.d.).

Over time, more lightweight virtualization methods emerged, functioning similarly to VMs but without fully emulating a physical computer. These are now commonly known as containers, with Docker (Docker, n.d.) being the most widely used container platform and Kubernetes (Kubernetes, n.d.) serving as the leading orchestration tool for managing and scaling containerized applications across distributed environments.

Several organizations, including cloud service providers (CSPs), have adopted virtualization in their cloud infrastructure. By providing computational resources, CSPs enable organizations to manage their cloud infrastructure efficiently.

A recurring challenge and major concern for CSPs is resource waste in their infrastructure, produced primarily from inefficient resource usage by users and the complexities of managing workloads in unpredictable environments. In early 2012, studies revealed low average CPU and memory utilization in Google's production clusters, at only 20% and 40% (Reiss, Tumanov, *et al.*, 2012), respectively. At that time, researchers reported similar findings in Amazon AWS EC2 with CPU utilization averaging between 7% and 17% (Liu, 2011). In 2018, Alibaba's Sigma and Fuxi had improved their resource utilization by achieving 38.2% CPU utilization and 88.2% memory utilization, but only considering online services (Guo *et al.*, 2019). When CSPs improve resource utilization, infrastructure costs are optimized, reducing waste from underutilized resources.

This thesis investigates issues related to inefficient resource utilization in cloud computing environments with a particular focus on the mechanism of overcommitment. Overcommitment involves CSPs allocating more resources than are available. While this strategy can increase resource utilization, it also introduces the risk of performance degradation if actual usage exceeds expectations. This thesis also explores how overcommitment strategies can achieve a balance between optimizing resource efficiency and preserving the quality of service (QoS).

1.1 Motivation

The primary motivation for studying overcommitment is to improve resource utilization within CSPs, as previously highlighted. Idle resources that are allocated but remain unused represent a significant resource waste for CSPs. By implementing overcommitment policies, these idle resources can be effectively utilized, leading to more efficient resource allocation.

The availability of CSP trace data, even in partial form, enables external researchers to study and gain valuable insights into CSP environments. The data support the design and testing of new models and policies by simulating how these strategies might perform in real-world cloud settings. In our research on overcommitment policies, these publicly available datasets provide a solid foundation for validating the resource allocation strategies using real-world data (Cortez *et al.*, 2017; Wilkes, 2011; Wilkes, 2020; Ding, 2018).

Overcommitment policies can also be applied in Kubernetes clusters, where clusters can manage nodes to be overcommitted with pods. Nonetheless, Kubernetes clusters lack built-in mechanisms for automated overcommitment. Manual configuration of the system can be time-consuming, especially in dynamic environments with frequently changing workloads. This requires constant adjustments by administrators, leading to wasted maintenance time. An automated system capable of overcommitting resources without administrator intervention could significantly improve administrative efficiency by reducing the time spent on cluster maintenance tasks (Rzadca *et al.*, 2020).

Another issue contributing to resource underutilization in cloud environments is that users frequently overestimate the resource requirements for their workloads. The accumulation of these overestimations results in significant resource underutilization. By understanding the extent of resource overestimation, overcommitment policies can be designed to mitigate the amount of unused resources (Rzadca *et al.*, 2020).

1.2 Objective

The objective of this research is to develop state-of-the-art overcommitment policies for CSPs. These policies are designed to minimize resource wastage by lowering the number of active machines, reducing QoS constraints to increase resource utilization, and consequently decrease infrastructure costs. To accomplish this objective, several key research questions related to overcommitment policies have been established:

(1) RQ1 - What overcommitment policies are currently employed in cloud computing?

(2) RQ2 - What artifacts and metrics can be utilized to evaluate the effectiveness of overcommitment policies?

(3) RQ3 - How can containers be categorized into classes to optimize the effectiveness of overcommitment policies?

(4) RQ4 - What approaches can be implemented to automate overcommitment policies?

1.3 Contribution

The main contributions of this thesis are:

(1) **Exploration of the state-of-the-art overcommitment policies.** The different types of overcommitment policies employed by CSPs are introduced and explored in Chapter 2, while a detailed literature review of overcommitment is covered in Chapter 3.

(2) **Filtering the dataset for evaluating overcommitment policies.** The dataset used to study overcommitment policies is explained in Chapter 6.

(3) **Development of a simulator to test overcommitment policies.** The methodology for the experiment is discussed in Chapter 4. The simulator is detailed in Chapter 5, and the experiment settings and results are presented in Chapter 7.

1.4 Thesis organization

This thesis is structured as follows:

- Chapter 2 explains key concepts necessary for understanding overcommitment.
- Chapter 3 presents a literature review on overcommitment in the cloud, organizing the information thematically to familiarize readers with the topic and explain the field.
- Chapter 4 introduces the methodology used in the research and the experimental decisions made for modeling the simulator.

- Chapter 5 covers the simulator, detailing the functionality of its components and the relationship between them.
- Chapter 6 describes the data pipeline process, beginning with an overview of the different data sources. These data sources are then compared to determine which are the most suitable for use. Next, the chapter outlines the data preparation process required for the simulator.
- Chapter 7 presents the experimental results obtained from various experiments to understand the environment with overcommitment policies.
- Chapter 8 summarizes the findings of the research and proposes future work, including potential improvements and areas for further exploration.

Chapter 2

Overcommitment

Before diving into the concept of overcommitment, it is important to introduce some fundamental concepts in cloud computing. After this introduction, the overcommitment problem is formulated, along with its associated advantages and challenges.

2.1 Cloud computing

Cloud computing is a shift from traditional on-premises computing. Traditional computing relies on hardware that can be expensive to purchase and maintain. Cloud computing eliminates the need for users to maintain hardware and instead provides a set of processing, storage, and networking resources for deploying computer workloads.

The paradigm of cloud computing offers several benefits. Users can scale resources as their customers demand, modify resources easily over the Internet, save physical space by eliminating hardware, and reduce costs by avoiding the need to acquire all the hardware from scratch. These advantages have led several companies to adopt cloud computing (Carroll *et al.*, 2011).

CSPs offer services to their users that can be categorized into three major types: **Infrastructure as a Service (IaaS)** provides users with on-demand access to their resources. **Platform as a Service (PaaS)** offers users hardware and software resources needed for cloud application development. **Software as a Service (SaaS)** delivers an application stack as a cloud service, including maintenance and management of everything from the underlying infrastructure to the application software (Google Cloud n.d.).

A contractual relationship is established between the user and the CSP. The CSP is responsible for delivering a predetermined level of quality of service (QoS). Failure to meet these obligations constitutes a violation of the contract. QoS is a metric used to determine whether the service is operating as specified in the contract. If the provider fails to meet the QoS requirements, it is considered a violation of the Service Level Agreement (SLA).

The QoS guarantees offered by CSPs are more difficult to violate than those used in this thesis. CSPs typically offer service uptime guarantees ranging from 90% to 99.99% of the total operational time (Baset *et al.*, 2012). In contrast, this thesis adopts stricter

QoS guarantees than those typically offered by CSPs. In particular, the QoS requirements in this study are designed to ensure that the total resource capacity of a machine is not exceeded.

Large public CSPs, such as Alibaba Cloud, Amazon Web Services (AWS), DigitalOcean, Google Cloud Platform (GCP), Huawei Cloud, and Microsoft Azure, compete with each other to attract customers to their platforms. These platforms must offer high-quality services and a user experience at reasonable prices. This competitive effort drives the platforms to provide excellent customer service while aiming to reduce costs and energy consumption to nearly zero (Cohen *et al.*, 2017).

Data from CSPs show that resources are often underutilized. Optimizing these unused resources could generate significant benefits. Even a 1% increase in resource utilization across systems operating at a scale of hundreds of millions could result in cost savings reaching millions. Researchers found that overcommitment results in cost reductions ranging from 1.5% to 17% in specific settings (Cohen *et al.*, 2017).

Virtualization is a technology that facilitates the creation of virtual instances of computer resources. The most well-known virtualization technologies are virtual machines and containers. A recent trend in cloud computing is the adoption of containers, which run lightweight application images directly on the host operating system, unlike virtual machines (VMs) that emulate an entire operating system. Containers are preferred due to their lightweight nature.

Overcommitment can occur in two scenarios: CSP settings and Kubernetes cluster settings. In the CSP scenario, a virtual machine (VM) is allocated to a physical machine (PM). In the Kubernetes cluster scenario, a pod is allocated to a node, according to Kubernetes terminology. For consistency in naming conventions throughout this thesis, the term "container" will refer to either a VM or a pod, while the term "machine" will refer to either a physical machine or a node. The first scenario corresponds to IaaS, whereas the second scenario corresponds to either IaaS or PaaS.

2.2 Resources types

The CSP operates an infrastructure designed to support the execution of workloads. This infrastructure is composed of physical machines, each containing distinct types of resources, with well-defined interactions occurring among them. This section provides an overview of the most commonly used resource types. The key resource types of a physical machine include CPU, memory, disk, network, and GPU. Although additional types of resources may exist, this section focuses exclusively on the most commonly utilized resources, and the less commonly used resource types are not addressed.

CPU (Central Processing Unit) is a computer resource for executing instructions and performing calculations for a system to operate. The speed of a CPU is measured in gigahertz (GHz) and represents the number of cycles the CPU can perform per second. Modern CPUs have multiple cores that can handle tasks simultaneously through parallel computing. Most CPUs also have multiple layers of cache memory (L1, L2, and L3) that enable instructions to have faster access to frequently used data.

Memory is a computer resource that stores data and instructions to enable quick access by the CPU. The resource is commonly referred to as Random Access Memory (RAM). It differs from disks such as hard drives or SSDs because it is volatile, meaning all stored data is lost when the system is powered off.

Disk is a computer resource that performs the reading and writing of data to persistent storage. The term "disk" originally refers to storage devices that utilize spinning platters, such as Hard Disk Drives (HDDs). The term is also used for storage devices without moving parts, such as Solid-State Drives (SSDs). Disk has several access methods: traditional read and write operations, memory-mapped I/O (mmap), direct I/O (read/write), and asynchronous I/O (AIO).

Network is a computer resource used for transferring data between physical machines. Network metrics are determined by bandwidth, typically represented in megabits per second (Mbps), which measures data transfer capacity, and latency, measured in milliseconds (ms), which is the time required for data to travel between the source and the destination. This includes the time taken for data to be sent from the source, received by the destination, and the response to be sent back to the source, often known as round-trip time (RTT).

GPU (Graphics Processing Unit) is a computer resource initially designed as an electronic circuit for digital image processing and computer graphics. As time has passed, GPUs have evolved to perform high-speed mathematical calculations through parallel processing, making them obligatory for tasks such as machine learning, scientific simulations, and other compute-intensive parallel applications.

The CSP allocates containers with the requested resource types on an available machine. Each resource type employs a distinct mechanism for overcommitment, determined by its specific functionality. This thesis focuses on CPU and memory resources because the trace data are limited to these two resource types.

2.3 Overcommitment

Overcommitment occurs when the total allocated amount of a resource type for containers exceeds the available capacity of that resource on a machine. In the literature, overcommitment is also referred to as oversubscription, overbooking, or multiplexing. On the one hand, when a machine is overcommitted, the containers inside the machine may experience overload, leading to potential SLA violations. On the other hand, when a machine is undercommitted, the machine has low resource utilization, resulting in wasted resources. Low resource utilization can lead to reduced profits, highlighting the importance of efficient resource allocation to improve both energy and cost efficiency.

A concept analogous to overcommitment in cloud computing can be observed in other industries, such as the airline industry, where it is referred to as **overbooking**. The problem in the airline industry can be described as follows: an airline has a plane with a fixed number of seats, and each passenger needs a seat during the flight. The objective is to maximize the number of occupied seats, as unoccupied seats represent wasted capacity that could have been assigned to paying passengers. At the same time, some passengers may cancel or fail to board their flight at the last minute.

The goal is to maximize profit while minimizing the occurrence of dissatisfied customers who might require compensation, such as hotel accommodations or rebooking on subsequent flights. This concept has a clear parallel to overcommitment in cloud computing. Unlike airline seats, resource allocations in cloud computing can often be represented as real numbers, allowing for fractional allocation and containers consisting of multiple resource types that the machine must simultaneously accommodate, unlike a seat in the airline analogy.

To illustrate the concept of resource overcommitment more clearly, an image of CPU overcommitment is provided. This concept can be extended to other types of resources as well. Figure 2.1 shows CPU usage over time for each container. The x-axis represents the elapsed time, while the y-axis represents the total resource allocation for each container.

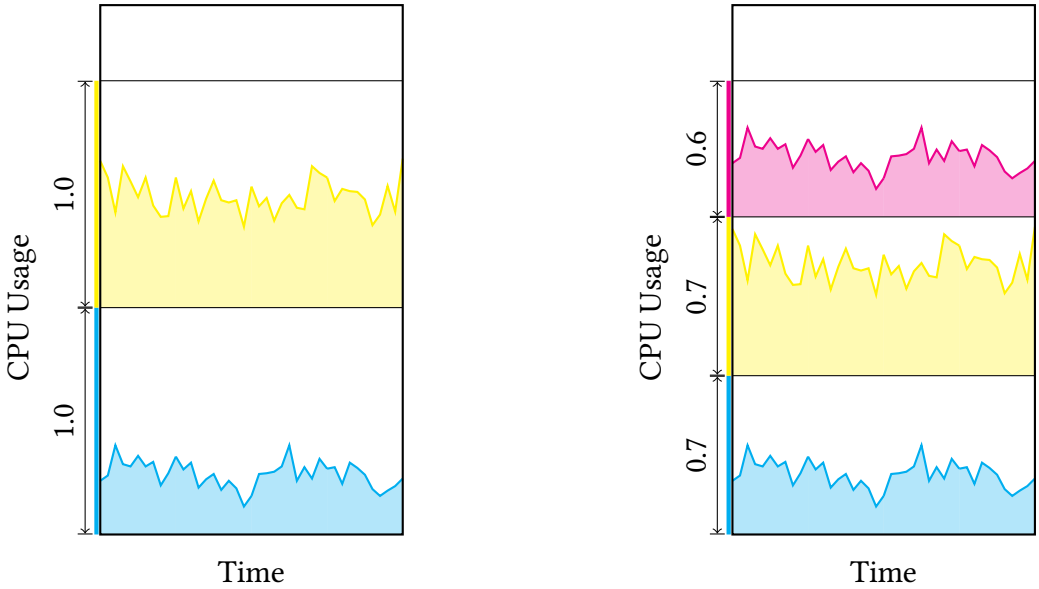

Figure 2.1: *Example of CPU overcommitment*

In the first image, two containers are shown utilizing CPU resources. Each container is allocated 1.0 CPU resource, but their combined resource usage does not exceed half of the available CPU capacity of the machine, leading to substantial resource underutilization.

In the second image, an additional container with CPU resources is introduced. Under the configuration shown in the first image, adding more containers would not be possible because the maximum allocation is limited to 2.0 CPU cores. However, since resource utilization can be estimated, the allocation for the previously added containers can be reduced to 0.7 CPU cores each, lowering the total allocation to 1.4 CPU cores instead of the original 2.0 CPU cores. The allocation adjustment makes it possible to allocate a new container with 0.6 CPU cores.

This scenario demonstrates a potential overcommitment situation, as the total allocated resources, including the container with 0.6 CPU cores, amount to 2.6 CPU cores, while the total machine capacity remains limited to 2.0 CPU cores.

2.4 Problem formulation

A mathematical model can be formulated for the overcommitment problem. A machine is defined as an entity with a fixed number of available resources, while a container is defined as an entity that executes a workload and requires resources. Containers are allocated to machines, and the objective is to minimize the number of machines used when allocating container resources within the available machine resources.

The problem is commonly known as **vector bin-packing**, for which no optimal solution exists. In this scenario, the CSP manages machines with finite resources, such as CPU, memory, disk, and network. The provider must assign containers to these machines, with each container characterized by its specific resources. The objective is to minimize the total cost, but calculating the cost is challenging because energy consumption depends on the usage of each resource by the machine. To simplify the problem, the cost metric is represented by the number of machines used: the fewer machines used, the lower the cost.

When the number of machines is reduced, containers are packed more tightly on each machine. This tighter packing can degrade the QoS of individual containers, potentially leading to performance slowdowns or even termination. A mathematical model similar to those presented in previous studies (Tchernykh *et al.*, 2020; Cohen *et al.*, 2017) can also be used to characterize this problem.

Consider a set of M containers, denoted by $C = \{C_1, C_2, \dots, C_M\}$, each of which runs different kinds of workloads. The container C_j is described by a value $r_{d,j}$, which represents the required amount of resource type d, and the total number of distinct resource types is D.

The CSP has a set of N machines, each with a resource capacity P_d for resource type d. Each container can be assigned to exactly one machine. This linear programming formulation describes the allocation problem.

$$
\begin{aligned}
\text{minimize} \quad & R = \sum_{i=1}^{N} y_i \\
\text{subject to} \quad & \sum_{j=1}^{M} r_{d,j} \cdot x_{i,j} \leq P_d \cdot y_i, i = 1, \dots, N, \; d = 1, \dots, D \\
& \sum_{i=1}^{N} x_{i,j} = 1, \qquad\qquad j = 1, \dots, M \\
& x_{i,j} \in \{0, 1\} \\
& y_i \in \{0, 1\}
\end{aligned}
$$

The decision variable y_i is equal to one if machine i is used, and zero otherwise. The decision variable $x_{i,j}$ is equal to one if container j is assigned to machine i, and zero otherwise. The objective is to minimize the cost R without SLA violations, but applying overcommitment eventually causes SLA violations.

The SLA violation function is determined by how much the machine's resource usage

exceeds its actual capacity. This measure can be evaluated by calculating the frequency of violations when usage exceeds the capacity or by calculating the magnitude of excess usage.

Solving the problem using integer linear programming is infeasible due to the exponential computation time, making it impractical for real-world CSP environments where resource allocation must occur in real time. Consequently, heuristic algorithms are employed to compute near-optimal solutions within a reasonable time frame. These near-optimal solutions are discussed in Chapter 5.

2.5 Resource overload

Resource overload occurs when a machine's usage approaches or exceeds its maximum capacity. Under these conditions, container performance becomes unpredictable, as each resource type exhibits different unusual behaviors. For example, when CPU resources are overloaded and a new CPU-intensive container is allocated, the performance of previously deployed containers can degrade, potentially causing slowdowns or making them unresponsive. Similarly, if memory resources become overloaded and a newly allocated container requests additional memory, the system may resort to swapping memory to disk. If memory demand continues to exceed available resources, the machine may forcibly terminate containers due to an out-of-memory (OOM) condition.

To mitigate resource overload, several techniques can be employed, such as stealing, quiescing, and live migration (Householder *et al.*, 2014). Stealing allows a master VM to reclaim resources from another VM to prioritize its own operations. Quiescing temporarily halts a VM's processes and creates a backup, allowing the VM to be resumed once resources become available. Live migration involves transferring the VM from Machine A to Machine B, reducing the resource usage on Machine A without interrupting or rebooting the VM. Additionally, CPU throttling is used when a container is close to its CPU limit, constraining the resource usage to prevent overload.

Another issue associated with memory resources, which can contribute to memory overload, is memory fragmentation. Memory fragmentation occurs when data are stored in non-contiguous blocks of memory. During periods of resource overload, fragmented memory may prevent a container from allocating the necessary resources, even when sufficient total memory is available. Memory resources are structured contiguously, but as memory is repeatedly allocated and deallocated, the number of contiguous blocks decreases. Virtual memory partially solves the problem by using paging techniques, which allow non-contiguous memory allocation (Halbuer *et al.*, 2024; Mansi and Swift, 2024).

2.6 Slack and automation

Resource limits for containers are usually configured manually by developers inside an organization. Developers often set these limits higher than the actual resource requirements of the container to prevent scenarios where a container experiences insufficient CPU or is terminated due to an OOM condition. The accumulation of these containers creates

extra slack in unused resources. This slack results in resource wastage, as it prevents the placement of additional containers that could otherwise utilize these excess resources (Rzadca *et al.*, 2020).

Developers may need to adjust resource limits as a container's requirements change during the project. Manually reconfiguring these limits consumes valuable developer time that could be better spent on other tasks. Toil is a metric that quantifies the human execution time required to perform a task (Rzadca *et al.*, 2020).

Google Autopilot presents a model that performs nearly automatic resource allocation. Developing a similar software solution for automatic resource allocation could significantly reduce the toil in cloud container management systems, such as Kubernetes clusters, by minimizing the need for manual configuration.

2.7 QoS reduction

After utilizing the unused resources from the extra slack, another possible optimization involves decreasing the QoS by throttling the CPU. This approach reduces unused resources but increases the total completion time (makespan). Containers host various types of applications, each with different requirements. For batch tasks where QoS is less critical, degradation can be applied without significant consequences. In contrast, for latency-sensitive services, QoS degradation can negatively impact user experience, potentially leading users to abandon the service or switch to competitors. Ensuring that each container operates efficiently on the CSP's infrastructure is essential for maintaining user satisfaction.

Defining effective policies for container allocation is crucial for minimizing resource waste. CSPs must gain a deep understanding of each container's specific characteristics and requirements to ensure user satisfaction and optimize costs. This task becomes increasingly complex due to the high number of containers, and identifying those suitable for QoS reduction is particularly challenging.

2.8 Overcommitment policies

Overcommitment policies require detailed information about containers. When a container is first assigned to a machine, its behavior and resource usage patterns are unknown. In such cases, the container is assumed to use the total allocated resources. The standard approach to this uncertainty is to adopt a conservative strategy. Over time, as the container runs and metrics are collected, its behavioral patterns become clearer. Once sufficient data are gathered, it becomes theoretically possible to identify and implement an overcommitment policy tailored to the specific requirements and behavior of the container.

Overcommitment policies can be categorized into machine-centric overcommitment and container-centric overcommitment.

2.8.1 Machine-centric

Machine-centric overcommitment occurs when a machine's physical resources are overcommitted by creating additional virtual resources, extending the total resource capacity beyond the actual physical limits. For example, if 100% of a machine's physical resources are allocated, adding an extra 20% of virtual resources increases the total capacity to 120%.

This strategy improves resource utilization because users frequently overestimate their resource requirements, leaving a significant portion of the allocated resources unused. The accumulation of these unutilized resources creates additional capacity that can be leveraged to allocate new containers, making it feasible to assign additional virtual resources.

An example of this approach is implemented by Microsoft Resource Central, which defines quotas for CPU overcommitment. In this approach, an additional 15% of virtual cores are allocated beyond the physical core capacity, effectively increasing the total resource capacity of the machine by 15%. This practice is exclusively applied to machines used for internal development. CSPs typically adopt a more conservative approach when managing users' containers to ensure system reliability and user satisfaction (Cortez *et al.*, 2017).

2.8.2 Container-centric

The container-centric approach involves reducing the resource limits for each container. By lowering these limits, more containers can be allocated to a single machine, thereby decreasing the total number of machines used.

The overcommitment rate employed by CSPs for their machines is usually based on the p-th percentile of resource usage or a predefined usage percentage interval. The overcommitment rate often depends on the type of task being executed. For batch tasks that do not require high QoS, containers can be overcommitted more aggressively. Conversely, for services, the overcommitment rate must be lower to avoid negatively impacting latency. There are two primary approaches to container overcommitment: static overcommitment rates or dynamic overcommitment rates.

Static

Static overcommitment is a standard method in which a fixed overcommitment rate is applied equally to all containers. This straightforward approach is widely implemented, including in systems like Google Borg (Bashir *et al.*, 2021).

Microsoft Resource Central applies overcommitment exclusively to non-production VMs. For these VMs, the overcommitment rate is set based on the 95th percentile of resource usage for each container (Cortez *et al.*, 2017).

Dynamic

Dynamic overcommitment adjusts the overcommitment rate based on the workload of each container. In Google Autopilot, the overcommitment rate is determined by the type

of task. For batch jobs, the rate is set based on the average resource usage, which is the 50th percentile. For online services, the rate is set at the 90th or 95th percentile, depending on the task's latency sensitivity.

For memory allocation inside Autopilot, the rate depends on the task's tolerance for OOM events. Minimal tolerance uses the maximum observed usage, low tolerance uses the 98th percentile of total usage, and intermediate tolerance takes the maximum between the 50th percentile of the observed maximum and the 60th percentile of total usage (Rzadca *et al.*, 2020), as shown in Table 2.1.

(A) *CPU overcommitment*

Batch job	P50
Normal Latency Service	P90
Low Latency Service	P95

(B) *Memory overcommitment*

Minimal OOM	Max
Low OOM	P98
Intermediate OOM	$\max(0.5 \cdot \text{Max}, \text{P60})$

Table 2.1: *Dynamic overcommitment*

Time-bound

Time-bound overcommitment is a special strategy exclusively for batch tasks and can fall into both the static and dynamic overcommitment categories, depending on the method used to determine T_{cpu}. Under this policy, the CPU allocation size is determined by $\frac{T_{cpu}}{R_{cpu}} \times A_{cpu}$, where T_{cpu} is the total CPU time from past runs, R_{cpu} is the reserved CPU time, and A_{cpu} is the actual number of CPU cores. The CPU allocation size function determines the number of CPU cores necessary to complete the task within the reserved CPU time, optimizing resource utilization while meeting time constraints. This method also improves the task latency, as the response time is lower due to more CPU resources being available (Kedia and Makagonov, 2018).

An example of a time-bound overcommitment policy can be found in Facebook's implementation of Apache Spark. Apache Spark is a distributed computing system designed for processing and analyzing large-scale data and executing batch jobs. The implemented policy leverages historical data from similar jobs executed over the past 10 days and utilizes the 90th percentile (P90) of CPU usage across all containers to estimate the total CPU time. By overcommitting resources and imposing hard limits on CPU usage, the policy ensures efficient utilization of CPU resources for non-latency-sensitive tasks while satisfying reserved time constraints (Kedia and Makagonov, 2018). Memory allocation follows the standard 99th percentile (P99) of memory usage.

2.9 Summary

This chapter introduced the fundamental concepts and context of cloud computing. The main computational resources used in cloud computing were presented: CPU, GPU, memory, disk, and network. This research focuses on two key resources: CPU and memory. After introducing these resource types, the overcommitment problem was formulated mathematically to explain the underutilization of computational resources. Some related ad-

vantages and challenges of overcommitment were explained, and several overcommitment policies were presented.

Overcommitment policies can be divided into two types: machine-centric and container-centric. The machine-centric strategy focuses on increasing the total machine resource size, as users generally allocate resources with extra slack. In contrast, the container-centric strategy focuses on reducing the total container resource size allocated to the machine.

In the container-centric method, each container's workload can be analyzed, allowing resources to be allocated based on the workload's characteristics. By categorizing containers into latency-sensitive and non-latency-sensitive groups, the QoS for non-latency-sensitive containers can be reduced to ensure better latency for those that are latency-sensitive. The container-centric method can be subdivided into static and dynamic. The static allocation size remains fixed for all container types, while the dynamic allocation size can change for each container. There is also a time-bound overcommitment strategy used exclusively for batch tasks.

Chapter 3

Literature review

A **regular cloud** is a cloud environment where resources such as network and disk I/O can be overcommitted. An **overcommitted cloud** takes a step further by also allowing CPU and memory to be overcommitted, in addition to the already overcommitted network and disk I/O (Baset *et al.*, 2012).

In a regular cloud, network and disk I/O resources are typically multiplexed among containers. **Multiplexing** is a mechanism that combines two or more signals or data streams from multiple channels into a single channel, integrating the channels among containers and allowing the same data stream to be used. The primary goal of an overcommitted cloud is to enable the overcommitment of CPU and memory resources as well, but these two resources are not as easy to overcommit as network and disk I/O and require other elaborate techniques to achieve overcommitment.

Using overcommitment presents an ethical dilemma: balancing the risk of being unfair to clients against the potential benefits of conserving resources that would otherwise go unused. If overcommitment significantly reduces the clients' QoS, it raises serious concerns about fairness and service reliability. Conversely, if it does not negatively affect QoS and successfully optimizes resource usage, it can be considered a responsible and efficient method to reduce resource wastage.

In an overcommitted system, security vulnerabilities can arise from improper isolation between containers hosted on the same machine. These vulnerabilities may lead to information leakage, where sensitive data from one container becomes accessible to other containers sharing the same machine. Such breaches pose significant risks, as sensitive information intended solely for the data owner may inadvertently be exposed, compromising the system's confidentiality and integrity (Kim *et al.*, 2014).

The literature review is structured thematically to highlight key areas of research, as illustrated in Figure 3.1.

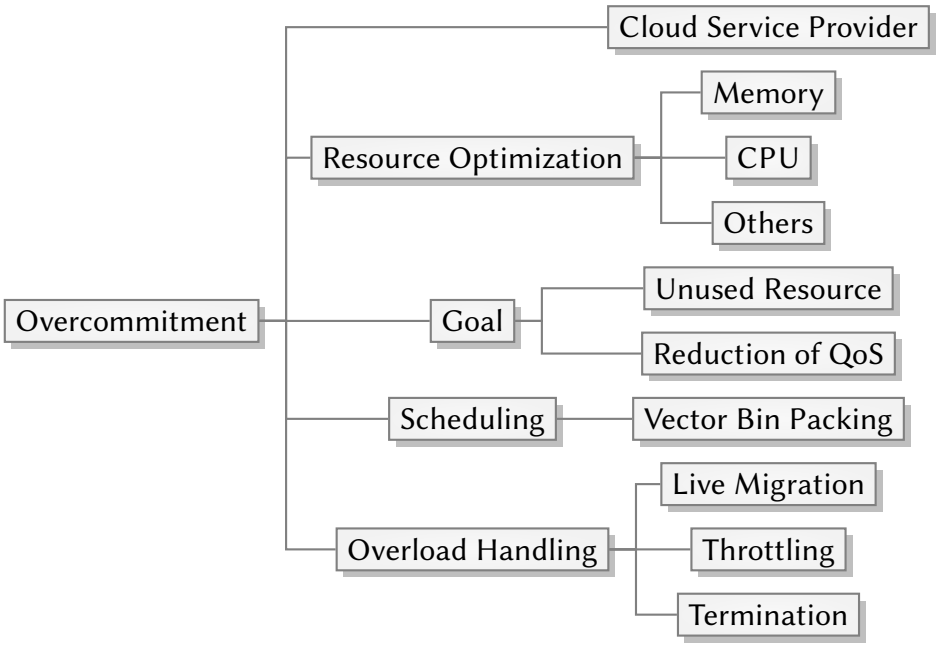

Figure 3.1: *Literature review*

The thematic diagram demonstrates how overcommitment involves several topics in cloud computing. The literature review begins by examining how CSPs overcommit their machines, focusing on strategies applicable to our research.

Each resource has a distinct method for overcommitment, as resources differ in their characteristics. The purpose of overcommitment has two main objectives: to utilize unused resources and to reduce QoS to prioritize resources for other containers.

The overcommitment strategies used in open-source cloud solutions are outdated. OpenStack, an open-source cloud solution created in 2010 by Rackspace and Anso Labs, uses default overcommitment ratios of 16:1 for virtual CPUs to physical CPUs and 1.5:1 for virtual RAM to physical RAM (OpenStack, n.d.). The ratio x:y indicates that for every y physical resources, x virtual resources exist.

In comparison, VMware vSphere provides a guideline for a 6:1 ratio of virtual CPUs to physical CPUs and a 1.5:1 ratio of virtual RAM to physical RAM. These values were established at a time when servers and codebases were not optimized for efficient CPU utilization. As the cloud landscape has evolved, new recommendations have emerged to reduce these ratios to 3:1 for virtual CPUs to physical CPUs and 1.25:1 for virtual RAM to physical RAM (Cloud Architect Musings, 2013).

The parameters used in both OpenStack and VMware can be considered outdated, and their overcommitment strategies need to be modernized to keep pace with advancements in hardware and software optimization. In Red Hat Enterprise Linux, a service named “Tuned” is available, which employs a mechanism for setting resource usage profiles for resource optimization but does not allow for parameter adjustments.

3.1 Cloud service provider

Google Cloud (Rzadca *et al.*, 2020), Microsoft Azure (Cortez *et al.*, 2017; Kumbhare *et al.*, 2021), and Alibaba Cloud (Z. Zhang *et al.*, 2014; Guo *et al.*, 2019) have been researched using cloud traces, analyzing how CSPs use the data to make scheduling and overcommitment decisions. These studies demonstrate how information about cloud environments is utilized, and such insights can help define effective overcommitment policies.

Google Cloud uses Autopilot, an automated task deployer designed to reduce the human toil required to configure resource needs. Google Autopilot employs an algorithm similar to reinforcement learning to allocate containers to machines. During system spikes, the algorithm uses a half-life technique to gradually adjust the container resource sizes, decreasing them over time. To monitor and manage spikes, the system calculates a lower limit and an upper limit to track its performance (Rzadca *et al.*, 2020).

Microsoft Azure uses Resource Central, which employs a machine learning algorithm to predict the resource utilization and differentiate between human and machine containers. In the infrastructure of CSPs, a more conservative approach is adopted for resource usage. Resource Central overcommits resources only to machines intended for development purposes (Cortez *et al.*, 2017).

Alibaba Cloud allocates batch tasks during night periods when CPU usage of services is lower, allowing batch tasks to utilize unused resources (Guo *et al.*, 2019).

3.2 Resource optimization

For disk I/O and network oversubscription, resources are multiplexed among the containers.

Memory oversubscription techniques include ballooning/reclamation, a method that allocates idle or unused memory pages for use. Compression is another technique where memory pages are compressed and stored on disk, though it has the drawback of requiring decompression later. Swapping is also a method that moves random memory pages to disk to free up physical memory. Ongoing research continues to explore and optimize these techniques to improve their efficiency and performance (Wang *et al.*, 2023).

Batch tasks can be scheduled alongside online services, utilizing CPU resources when resource usage from online services is low (Guo *et al.*, 2019). Another innovative approach to leveraging underutilized resources involves using GPU resources during the idle phases of deep learning processes to perform image rendering for live gaming services (W. Zhang *et al.*, 2022).

3.3 Overcommitment

For overcommitment, Datos separates jobs into two types of queues: a normal queue and an overcommitted queue. The normal queue executes jobs at standard priority, while the overcommitted queue assigns lower priority to its jobs, throttling them to prioritize the normal queue based on the host machine's overload level (J. Chen *et al.*, 2018).

QoS reduction can also be applied without using queues, as in Datos, by directly throttling the container for tasks that do not require low latency. In such cases, a slow-down in service might be acceptable, but this approach carries the risk of impacting user satisfaction if the product experience deteriorates to the extent that customers abandon the system.

Several overcommitment policies are explained in prior research (Bashir *et al.*, 2021). In the terminology used, these policies are referred to as peak predictors. The Borg-default predictor calculates the sum of container allocations and applies a fixed rate to the total sum, aligning closely with the machine-centric overcommitment method. The Resource Central-like predictor calculates the sum of container allocations based on the 95th percentile, similar to the container-centric static overcommitment method. The paper also introduces an N-sigma predictor that uses a Gaussian approximation to estimate the resource usage (Janus and Rzadca, 2017). Additionally, studies have explored overcommitment in the context of CPU usage distributions for containers. In these studies, container CPU usage is modeled using either a truncated Gaussian or a Bernoulli distribution. Synthetic data are generated based on these models, and then a fixed overcommitment rate is applied to run the experiment (Cohen *et al.*, 2017).

3.4 Vector bin-packing

Vector bin packing involves packing n-dimensional vectors into n-dimensional boxes. In earlier research, this problem was also referred to as multidimensional bin packing, a term that can easily be confused with the task of packing n-dimensional boxes into an n-dimensional box.

An example illustrating the difference between vector bin packing (VBP) and multidimensional bin packing (MBP) in two dimensions is provided in Figure ??.

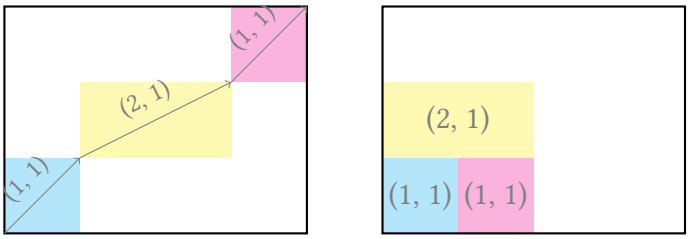

Figure 3.2: *Difference between VBP and MBP*

The allocation problem can be formulated as vector bin-packing, where the CSP wants to allocate containers represented by resource demand vectors into a box that corresponds

to the available resources of the machines.

When the dimension is two or greater, the vector bin packing problem becomes NP-hard. There is no Asymptotic Polynomial-Time Approximation Scheme (APTAS) available for dimensions $n \geq 2$ (Ray, 2024). APTAS refers to a polynomial-time algorithm capable of producing a solution within a factor of $(1+\epsilon)\cdot OPT$, where OPT is the optimal solution and $\epsilon > 0$. Since an optimal algorithm does not exist, different strategies have been explored to establish bounds for vector bin packing in lower dimensions (Kulik *et al.*, 2023), but due to the lack of straightforward optimal algorithms, many practical applications rely on simple heuristic algorithms.

Research conducted by Microsoft evaluated several heuristics for solving the vector bin packing problem (Panigrahy *et al.*, 2011). More recently, researchers have extensively tested vector bin packing algorithms across various scenarios and have introduced the **Vectorpack** library, specifically designed for evaluating these algorithms (Mommessin *et al.*, 2025).

3.5 Overload mechanism

Overcommitting resources can significantly impact the performance of a machine. When resources are heavily utilized, the machine may become overloaded. In this state, where the resources operate close to their maximum capacity, tasks running on the overloaded machine frequently experience performance degradation.

When a machine becomes overloaded, several mitigation strategies can be implemented. One approach is live migration, which moves containers from one machine to another to balance the load (Forsman *et al.*, 2015; Jiang *et al.*, 2021). Another method is resource throttling, which restricts container resource usage to mitigate overload conditions. In extreme cases, terminating a container may become necessary to free up critical resources and maintain normal operation for the remaining containers (Rzadca *et al.*, 2020).

3.6 Comparison with existing simulator

Table 3.1 provides a comparison of existing overcommitment simulators from both cloud simulation and the literature with the current work. Integrating the cloud simulation overcommitment components into a unified framework presents significant challenges. The research literature offers few overcommitment simulators, and those available typically require adjustments for reproduction or are closely tied to specific cloud trace datasets and technologies.

[1] The scheduler only shifts the values and allocates them in a first-fit manner.

[2] The focus of the cloud simulator is not overcommitment, and it has limited functions for overcommitment.

Simulator	Open-source	Overcommitment	Overload handler	Scheduler
Our work	✓	✓	✓	✓
Bashir et al.	✓	✓	✗	✓ [1]
CloudSim	✓	✓ [2]	*?*	✓
Filelis et al.	✗	✓	*?*	*?*

Table 3.1: *Current work comparison*

In Table 3.1, the only open-source overcommitment simulator besides Clovers, supporting multiple overcommitment techniques is the one developed by Bashir et al., but it doesn't have advanced scheduling methods or overload handling, and is restricted to the Google Borg dataset. In comparison, CloudSim provides a robust cloud simulator framework but offers only limited overcommitment policies, and is referred to as overbooking or time-multiplexing.

3.7 Summary

This chapter begins with an overview of the concept of overcommitted clouds and discusses the associated ethical and security concerns. The literature review is organized thematically, presenting a historical evolution of overcommitment rates. Research from cloud providers is then examined, with each approach offering valuable insights for future studies.

The next section introduces investigations into optimizing individual resources and discusses various overcommitment studies. Subsequent sections explore vector bin-packing research and examine overload handling strategies described in the literature. Finally, the chapter compares existing overcommitment simulators, highlighting their respective strengths and limitations.

Chapter 4

Methodology

This chapter describes the research methods used to study overcommitment policies, always focusing on the research goals and questions. The research goals involve understanding how these policies can support CSPs and cluster managers in the development of more efficient resource utilization while also creating an efficient artifact that provides a fast feedback loop for experimentation.

The goal of overcommitment policies is to reduce costs by minimizing the number of machines used and decreasing the number of SLA violations. Several artifacts can be used to test these overcommitment policies. These artifacts for evaluating algorithms or policies in experiments can be separated into: In-situ, Simulation, Emulation, Benchmarking, and Formal Methods (Calheiros, 2010).

"In-situ": The term originates from Latin, meaning "in the original place". This method involves conducting experiments directly in the real environment. Using this approach requires careful consideration of potential risks, as a crash during the experiment could affect other programs running on the same machine, which can be problematic if those programs serve real clients.

Simulation: This method involves creating a virtual environment that replicates the mechanisms of a real environment using a model, allowing systems to be tested, broken, or modified without risking the real infrastructure.

Emulation: This method replicates processes step by step, matching how each process occurs in a real environment while running in a virtual-controlled setting. This approach allows testing mechanisms that the simulation may not capture, showing effects in processes that arise from the system's internal operations.

Benchmarking: This method utilizes a collection of programs intended to evaluate the execution of a cloud computing environment.

Formal methods: These methods apply a rigorous mathematical formulation for the specification, development, and verification of software and hardware systems.

The approach of utilizing artifacts in experiments extends to cloud computing settings, where each artifact is evaluated using predefined attributes. The effectiveness of each

artifact is measured based on attributes such as reproducibility, extensibility, applicability, revisability, scalability, development effort, modeling effort, and accuracy (Calheiros, 2010). Table 4.1 provides a comparison between the artifacts and the attributes.

Reproducibility: This ensures that the artifact can replicate the experiment and consistently yield the same results.

Extensibility: This ensures that the artifact can adapt to new experiments using the same code base with minimal modifications.

Applicability: This ensures that the artifact uses a dataset that is realistic and representative of the experiment.

Revisability: This ensures that the result of the artifact can be verified by checking metrics and identifying errors.

Scalability: This ensures that the artifact can scale the experiment to different levels by adding new resources or modifying parameters as required.

Development effort: This measures the difficulty of developing the system required to utilize the artifact.

Modeling effort: This measures the difficulty of modeling the system required to utilize the artifact.

Accuracy: This evaluates the closeness between the results of the artifact and those from a real environment, assessing the precision of the outcomes.

	In-situ	Simulation	Emulation	Benchmarking	Formal methods
Reproducibility	low	high	high	low	high
Extensibility	low	high	high	medium	low
Applicability	low	medium	medium	low	medium
Revisability	low	high	high	low	low
Scalability	low	high	medium	low	low
Development effort	low	high	low	high	high
Modeling effort	low	high	low	high	high
Accuracy	high	low	high	high	low

Table 4.1: *Artifact for experiments*

4.1 Design

"In-situ" and emulation artifacts cannot be executed repeatedly because rerunning the entire cluster is computationally expensive. Each re-execution requires all programs in the cloud environment to run again, consuming significant computational resources. Executing the real environment after multiple simulator runs provides an effective way to verify that the simulation accurately reflects real-world behavior. Hidden variables, such as context switching, overload conditions, or overlooked assumptions, often become more apparent during these tests.

The main artifact employed in our experiment is a simulation. The most popular cloud simulation tools are CloudSim, widely utilized by researchers, and CloudSimPlus, a modified version with simpler syntax and efficient performance, but both tools lack support for overcommitment policies and heterogeneous resources (Filelis-Papadopoulos *et al.*, 2018).

Overcommitment is a common practice in modern cloud systems and should be integrated into cloud simulation. Due to the complexity of projects like CloudSim, it is challenging to identify which features require modification and understand their interactions. To address these limitations, a new simulator was developed from scratch using Rust, inspired by CloudSimPy (Li and Hu, 2019) as well as CloudSim and CloudSimPlus.

The workflow of the overcommitment experiment is illustrated in Figure 4.1, describing the steps and processes involved in the experiment.

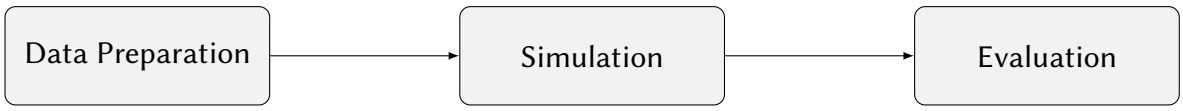

Figure 4.1: *Methodology design*

During the data preparation phase, information is extracted from the CSP trace dataset, selecting only the data relevant to the experiment. After extraction, the selected data is processed, and the simulation is executed. The data extraction is slow when performed individually for each container. The speed-up achieved using batch processing during data extraction is discussed in detail in Chapter 6.

During the experiment, several critical design decisions were made. For example, the dataset includes a wide variety of workload types, and it is necessary to reduce the number of workload types processed by the simulator. To simplify the analysis and focus on the experiment's objectives, the workload types are narrowed down to two categories: batch tasks and service tasks. Service tasks are latency-sensitive, while batch tasks are not. This reduction optimizes data processing, since the majority of tasks fall into these two categories, and provides a clearer understanding of how overcommitment policies impact these tasks.

The simulator selects a subset of containers, each containing a list of containers executed in the real environment. When machine information is unavailable in the dataset, selection is made directly at the container level. To ensure reproducibility and consistency across multiple experiment runs, a fixed ChaCha8 hash is used for each run. This hash value initializes the random generator, maintaining identical containers for every run. These machines and containers are then used by the simulator to evaluate several overcommitment policies and scheduling algorithms. At the end of the simulation, the simulator generates metrics such as the total number of machines utilized and the number and magnitude of SLA violations. For more detailed information about each step in the process, refer to Chapters 5 and 6.

To handle overload, researchers typically employ live migration to relocate containers between machines. While incorporating live migration into the experiment could provide valuable insights, this approach would significantly increase the complexity of the

simulation. As a result, overload handling in this experiment is restricted to throttling or terminating containers when predefined conditions are met.

By design, the simulator does not fully replicate cloud-environment behavior, omitting factors such as the overhead of CPU context switching, memory operations related to ballooning and paging, and system overload effects. System overload effects refer to the changes in resource usage depending on how users react to overload. For example, users may stop using the service due to slow performance, leading to a reduction in CPU usage, or users might repeatedly reload and retry accessing the service, which can intensify the overload and increase CPU usage.

4.2 Summary

This chapter described the methodology adopted in the thesis. The chapter began by presenting a list of artifacts and their attributes to clarify how the artifacts could be compared. In designing the experiment, a simulation was used to test the cloud environment without the need to repeatedly run computationally expensive "in-situ" and emulation artifacts.

Overcommitment is a common feature in modern cloud systems and is necessary to integrate into the cloud simulator. A new simulator was developed to evaluate overcommitment policies and other features associated with overcommitment. The workflow of the experiment was described in detail, covering the collection and extraction of the data, the overload mechanism applied to the machine, and how the experimental metrics were used for assessment.

Chapter 5

Cloud simulator

A cloud simulator is a tool designed to replicate the cloud computing environments, reproducing the behavior and functionality of a real cloud system. It enables researchers and cloud administrators to model and analyze cloud infrastructure in a controlled environment, providing insights into cloud performance statistics under specific scenarios.

The simulator offers the advantage of testing a cloud environment without the need to deploy actual cloud resources. It allows researchers and cloud administrators to simulate the execution of services and applications inside a container without real cloud infrastructure. As a result, users can experiment and analyze system behavior without incurring the costs or risks associated with infrastructure failures.

5.1 Comparison of different cloud simulators

In the cloud simulation space, CloudSim is the most popular and widely used cloud simulator among researchers and is an open-source framework written in Java, designed to simulate events in a cloud environment. CloudSimPlus is another important simulator with a simpler syntax and more efficient performance than CloudSim.

CloudSim offers a range of features for running cloud simulations and benefits from a growing community that continuously improves and expands its capabilities (Andreoli *et al.*, 2024). Despite these advantages, CloudSim also has limitations. Some features, such as several overcommitment policies and overload handling, are not supported to my knowledge.

5.2 Clovers — cloud overcommitment simulator

Clovers is a cloud simulator based on discrete-event handling, implemented in the Rust programming language as part of a thesis exploring overcommitment policies. Introduced in 2015, Rust focuses on memory safety to minimize bugs and avoid undefined behavior through language design. Memory safety in Rust is enforced by eliminating uninitialized variables, preventing null pointers, and enforcing strict mutability rules through compile-time verification.

Benefits of Rust include strong memory safety guarantees and a powerful compiler capable of detecting complex issues, such as data races and buffer overflows. Rust also has certain drawbacks, such as longer compilation times, a relatively steep learning curve, and minor limitations associated with its trait system.

CloudSim, the most popular cloud simulator, is implemented in Java. The Java language offers several advantages, such as an extensive ecosystem and platform independence across operating systems. There are also drawbacks related to the Java Virtual Machine (JVM), such as high memory allocation and reliance on garbage collection, which can negatively impact performance. Furthermore, application execution on top of the JVM may introduce additional latency. Other concerns include potential null pointer exceptions, which can cause unexpected program termination, and out-of-memory errors resulting from inadequate memory allocation.

5.3 Components of the simulator

The cloud simulator consists of the following components:

1. **Engine**: The engine is the heart of the simulator and processes cloud events at every timestamp. Events include tracking resource usage for overload management and handling the insertion and deletion of containers and machines.
2. **Overcommitment Policy**: This component manages resource overcommitment. The static policy applies the same p-th percentile value across all containers or machines, while the dynamic policy adapts the p-th percentile based on container type, using a higher value for batch tasks and a lower value for services.
3. **Scheduler**: Responsible for allocating virtual resources such as CPU and memory to containers, while determining appropriate machines to host each container.
4. **Overload Handler**: Detects resource overload conditions and communicates these events to the engine. The engine processes these events and performs appropriate simulation actions, such as throttling or terminating processes.
5. **Evaluator**: Measures the results of an experiment on the simulator.
6. **Resource Usage Interface**: Provides graphical representations of resource usage metrics, helping users to visually interpret machine and container resource usage.

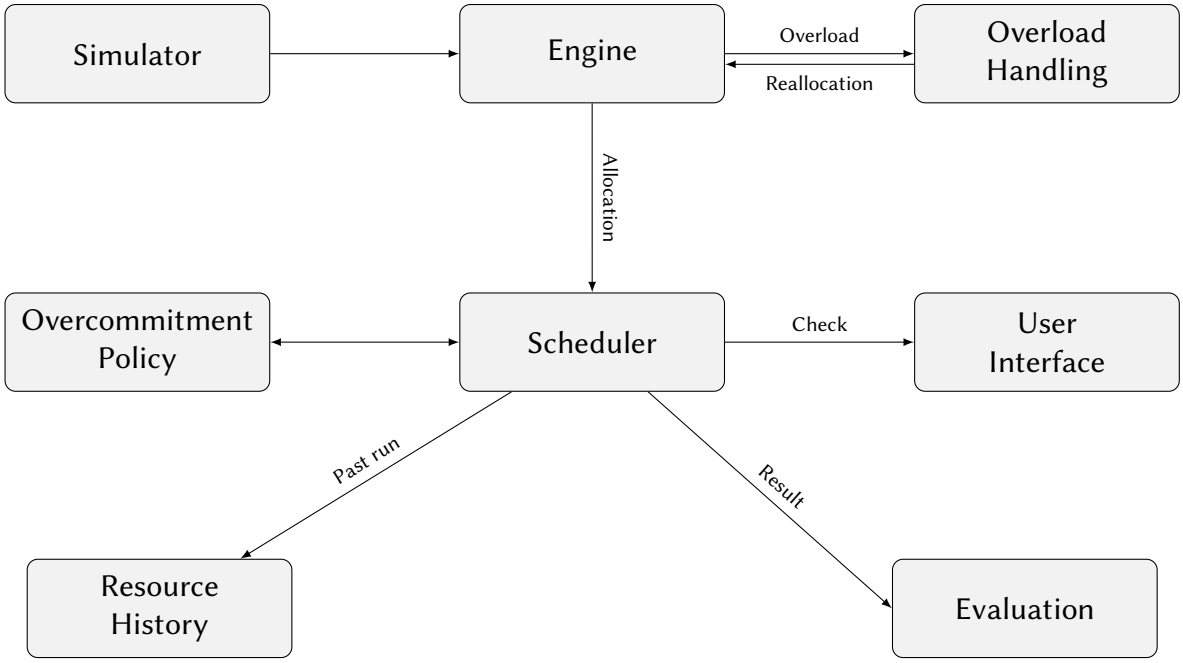

Figure 5.1: *Cloud simulator diagram*

5.4 Engine

The simulator consists of multiple interdependent components, with the engine serving as its core. When an overload is detected, the engine triggers the overload handler. The overload handler evaluates the situation and determines whether to throttle or terminate a container to resolve the issue. When a new container is introduced, the engine relies on the scheduler and the overcommitment policy selector to determine which machine is appropriate for allocation and which overcommitment policy to apply.

After the simulation completes, the evaluator has the aggregated results from data collected during overload events. Alternatively, users can use the visual interface to observe the execution step by step. The engine processes the time series data for each container, calculating resource usage at every time step. The next section presents detailed implementation information for each simulator component.

The CSP trace collector gathers resource usage data at fixed time intervals in a real-world environment. While recording data every second would be ideal, doing so is resource-intensive and impractical. Frequent profiling can interfere with other containers, affecting the results. To address these challenges, the CSP trace collector selects an appropriate capture interval. For example, the Google Borg trace records data every 5 minutes, generating approximately 1 TB of information monthly. Any missing data points are completed using the most recent recorded data.

By design, the simulator model does not entirely reflect real-world scenarios. For example, container behavior becomes unpredictable when a machine reaches overload conditions, causing resource usage to fluctuate depending on the running container. The simulator assumes that the container maintains the same resource consumption during overload conditions.

5.5 Overcommitment policies

Overcommitment policies can be categorized as follows:

Machine-centric: The machine increases its physical resource capacity by adding a virtual resource equivalent to $x\%$ of the physical resource.

Container-centric — p-th percentile Static: Resource allocations for all containers have their resource allocations reduced to the p-th percentile of their workload.

Container-centric — p-th percentile Dynamic: Different types of tasks, such as batch tasks and online services, have their resource allocations reduced to a different p-th percentile of their workload.

Container-centric — Time-bound: This policy applies exclusively to batch tasks. CPU throttling manages the execution time for these tasks based on the calculated CPU allocation size. The CPU allocation size is determined by $\frac{T_{cpu}}{R_{cpu}} \times A_{cpu}$, where T_{cpu} is the total CPU time from past runs, R_{cpu} is the reserved CPU time, and A_{cpu} is the actual number of CPU cores.

5.6 Scheduler

The scheduler assigns containers to machines based on defined criteria to determine the placement of each container. Key considerations for scheduling are discussed in (Bittencourt *et al.*, 2018), and similar principles and challenges apply to overcommitment policies in cloud scheduling. The scheduler addresses tasks using a vector bin-packing approach.

5.6.1 Scheduling algorithms

Container scheduling is processed in the order of its allocation times. To assign containers to machines, various heuristics can be employed, each targeting specific objectives. The primary goals are to minimize SLA violations and reduce the number of machines utilized.

To minimize the number of machines used, the problem focuses on tightly packing machines to maximize the utilization of all available resources. Before introducing the scheduling algorithms, it is essential to establish some key definitions:

- **Vector of machine availability**: Calculated as the difference between the machine's total resource vector and its resource usage vector.
- **Vector of container peak usage**: Calculated by taking the maximum value across each resource dimension in the container's usage vector.

The scheduling algorithms implemented in the simulator are as follows:

First-fit: The scheduler selects the first machine with sufficient available resources and allocates the container to it. A variation, **First-fit Decreasing**, assigns a score to each

container, sorts them in descending order by their scores, and then allocates to the first machine with sufficient resources.

Best-fit: The scheduler uses a heuristic that sorts machines by their available resources and allocates the container to the machine with the smallest available resources that can still allocate the container. For multiple types of resources, the scheduler calculates a score as $\sum_i w_i \cdot (a_i(x) - d_i(v))$, where $a_i(x)$ is the availability of resource i on machine x, $d_i(v)$ is the container's requirement for resource i, and w_i is the weight of the resource i, representing its scarcity (Hadary *et al.*, 2020).

Tetris / Dot product: This heuristic calculates the dot product between the vector of machine availability and the vector of container peak usage. The container is allocated to the machine with the highest dot product value (Grandl *et al.*, 2014).

L2 norm / Euclidean distance: This heuristic calculates the distance between the vector of machine availability and the vector of container peak usage, under a specified norm (e.g., L2 norm or Euclidean distance) rather than using the dot product (Panigrahy *et al.*, 2011).

5.7 Overload handler

When a server operates under high utilization, it may encounter sudden spikes in resource usage. These spikes can be triggered by various factors, such as temporary hardware failure or a sudden Distributed Denial of Service (DDoS) attack, resulting in a sharp surge in incoming requests.

The optimal solution is to allocate additional resources to latency-sensitive workloads, ensuring that servers do not experience performance degradation due to insufficient resources. Alternatively, implementing an overload handler mechanism can effectively manage high-load situations.

In the simulator, when a machine becomes overloaded, specific tasks must be either throttled or terminated to mitigate the load. The simulator prioritizes terminating tasks with highest resource comsuption to reduce the overall machine usage. While alternative strategies, such as live migration, are available for handling overload, this thesis focuses on the overcommitment policy by prioritizing throttling and termination mechanisms commonly employed in real cloud environments (Hadary *et al.*, 2020; Rzadca *et al.*, 2020).

In real environments, overload policies for containers are typically separated into two categories: **soft limits** and **hard limits**. Furthermore, different resources often require unique mechanisms for managing overload, as summarized in Table 5.1.

In a simulated environment, a new type of overload policy can be introduced: **no limits**. Under this policy, no containers are terminated during overload situations, and instead the evaluator is notified about the issue.

Overload Handler	CPU	Memory
No Limit	Machine CPU unlimited	Machine memory unlimited
Soft Limit	Machine CPU limited Container CPU semi-limited Throttle tasks	Machine memory limited Container memory semi-limited OOM kills the biggest process
Hard Limit	Container CPU limited Throttle task	Container memory limited OOM kills the process

Table 5.1: *Overload handling method*

For the memory policy, when a container is set to a soft limit, the simulator waits until the machine becomes overloaded before terminating the container. In contrast, with a hard limit, the simulator immediately terminates the container once its memory usage exceeds the specified limit.

For the CPU policy, when a container is set to a soft limit, the simulator waits until the machine becomes overloaded before restricting the container's CPU usage. In contrast, with a hard limit, the simulator immediately throttles the container's CPU resources upon exceeding the specified limit, ensuring that the container does not surpass its allocated resources.

For CPU hard limit, the throttle mechanism requires a propagation model to manage unused CPU resources, since any unused CPU capacity must be carried forward to the next time step. Two methods are proposed for handling this propagation:

1. **Accumulation Method**: All unused CPU resources are accumulated and fully propagated to the next time step.
2. **Queue Method**: A queue stores unused CPU resources, with only the first value from the queue propagated to the next time step.

In the simulator, at each time step, the overload handler checks for any violation of resource limits. The simulation collects information, ensuring that, at every timestamp, each machine in the cluster reports whether container or machine violations have occurred.

5.8 Evaluation metrics

Two key metrics are used to evaluate the overcommitment policy: the number of SLA violations and the number of machines used. For SLA violations, two metric collection approaches are defined: measuring the magnitude by which the total resource capacity is exceeded and counting the frequency of occurrences in which resource limits are violated.

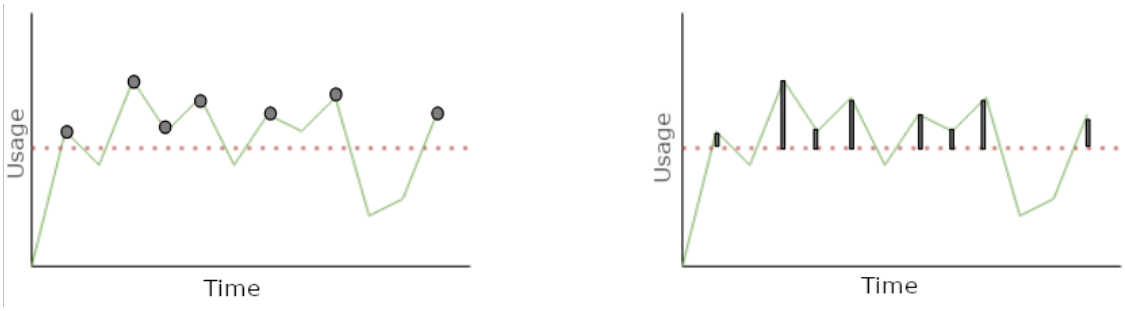

Figure 5.2: *Evaluation method*

The figure illustrates the evaluation of SLA violations. The x-axis represents time, while the y-axis represents CPU or memory usage. The red dotted line marks the limit at which SLA violations occur. Gray dots represent the frequency of SLA violations, while the black bars indicate their magnitude.

The metrics for SLA violations are defined as follows:

$$SV1 = \sum_{i=1}^{N}(\max(P_i - \sum_{j=1}^{M} p_j \cdot x_{i,j}, 0))$$

$$SV2 = \sum_{i=1}^{N}([\max(P_i - \sum_{j=1}^{M} p_j \cdot x_{i,j}, 0) > 0])$$

Definitions:

- P_i**:** The resource capacity of machine i.
- p_j**:** The resource usage of container j.
- $x_{i,j}$**:** A boolean variable indicates whether container j is allocated to machine i.
- []**:** A boolean operator that returns 1 if the condition inside the brackets is true and 0 otherwise.

The evaluation metrics are visualized in Figure 5.2. Quantifying SLA violations is a challenging task since there is no universally accepted method to measure a container QoS. Instead, these metrics provide an estimation of container behavior rather than an exact representation of real-world performance.

5.9 Implementation

The implementation of the overcommitment simulator can be found at https://www.gitlab.com/cloudsimrs/clovers.

5.9.1 Simulator implementation

The engine serves as the core component of the simulator, functioning as a state machine with two primary states: **event processing** and **load calculation**. These states alternate at each timestamp.

In the event processing state, cloud events are processed, resulting in changes to machines and containers. In the load calculation state, the load of the cloud environment is evaluated to identify system overloads.

Cloud event

The cloud system handles various events, including the add and remove operations for containers and machines. It also processes overload handling events, which may involve throttling or terminating containers to control resource usage. The evaluator can disable resource overload handling for testing purposes.

Overcommitment

To enable more flexible resource allocation, the concept of virtual resources is introduced. Virtual resources represent the modified values of physical resources for both machines and containers. These virtual resources are used to track and manage total resource usage, providing a more dynamic approach to updating resource limits.

Overload handler

After processing all events for a timestamp, the overload handler verifies the resource usage. During this phase, the engine examines every machine and container to identify any instances where resource usage exceeds the defined limits. The engine then reports any detected overload to the evaluator for experiment analysis.

Evaluator

The evaluator monitors the performance of the simulator by calculating several metrics at each timestamp, especially when overloads occur. It provides key insights into system performance and offers valuable feedback for optimizing the cloud environment.

Resource usage interface

The resource usage interface aggregates CPU and memory usage from all containers and displays resource utilization. The interface is implemented using Plotters, enabling a visual representation that helps users effectively monitor and analyze resource utilization.

Component testing

To ensure the functionality and maintainability of the cloud simulator system, each component is tested using a unit test. Rust provides a built-in library for testing the code, eliminating the need for external dependencies.

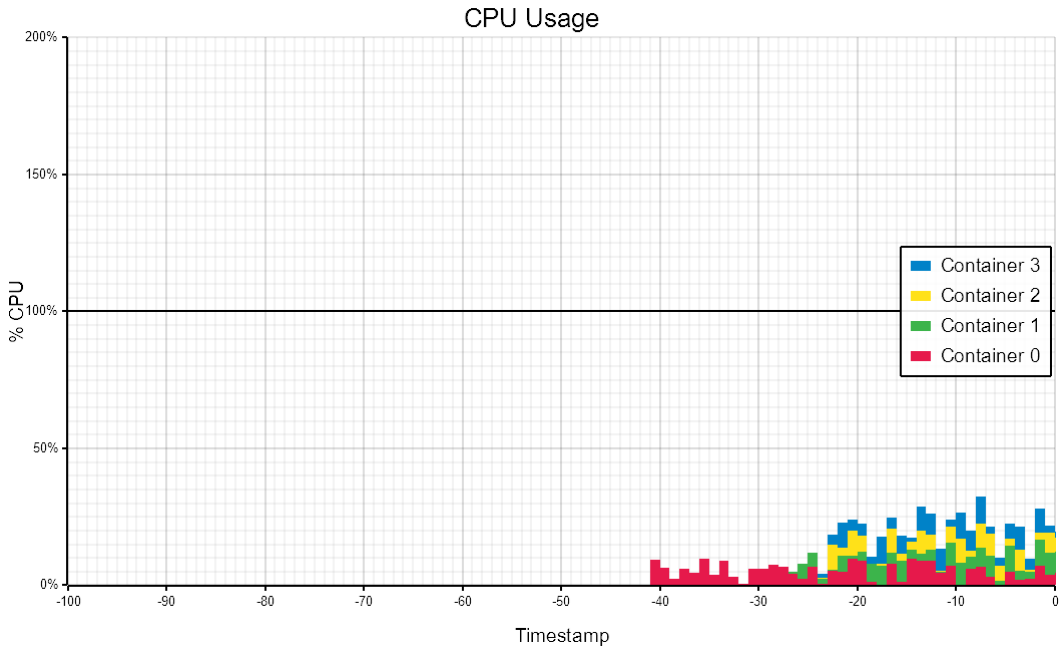

Figure 5.3: *Simulator — CPU usage*

Resource history

The resource history component in the cloud simulator stores historical utilization data and is responsible for calculating the p-th percentile resource usage for each container. Further details regarding the calculation of the p-th percentile are present in the following subsection.

5.9.2 Calculating the percentile of resources

A key challenge is determining how to calculate the p-th percentile of resource usage, as the process is not straightforward. This subsection describes the methods used to address this challenge.

The **percentile** represents the value below which a specified percentage of the data fall. To calculate the p-th percentile, the data are first sorted in ascending order. The value at the k-th position is then identified, where $k = \frac{p \cdot (n+1)}{100}$ and n represents the total number of data points. The value of k is rounded to the nearest integer.

For calculating the percentile of resource usage, two approaches are utilized:

- **Full data**: This approach considers all resource usage data in the resource history without restrictions.
- **Time window data**: This method focuses on data within a specific time window in the resource history. It is suitable for cloud computing environments where container resource usage fluctuates over time due to changes in workload or user behavior.

The naive approach to calculating the percentile involves sorting the data and directly identifying the number at position k. Although straightforward, this method becomes

computationally expensive when handling large datasets. In contrast, a more efficient approach uses a specialized data structure to dynamically maintain the k-th number of the data. This significantly improves performance and scalability, making it more suitable for real-time cloud computing applications.

Full data

To efficiently calculate the percentile of resource usage, a priority queue can be utilized. A priority queue supports three primary operations:

1. **Insert number**: Add a number to the priority queue.
2. **Find the maximum number**: Return the maximum number in the priority queue.
3. **Remove the maximum number at the top**: Remove the maximum number from the top of the priority queue.

The process begins by inserting numbers into the priority queue until the size reaches $k = \frac{p \cdot (n+1)}{100}$. After this point, the algorithm continues inserting numbers using operation 1 (insert) as data arrive. If the size of the priority queue exceeds k, the maximum number is removed using operation 3 before adding the new number.

During this process, the total number of data points n changes dynamically as values are added and removed. At any given time, the maximum number at the top of the priority queue (queried using operation 2) represents the p-th percentile of resource usage. This approach ensures efficient and dynamic percentile calculation.

Time window data

The default time window is set to the value T. When the time window shifts beyond T, the algorithm removes the data from the beginning of the window and inserts new data at the end. If normal calculations are used, the data must be sorted each time a number is added or removed, resulting in a time complexity of $\mathrm{O}(N \cdot T \cdot \log T)$, where N is the size of the data and T is the size of the time window. However, this method is inefficient and slow for large data sizes.

A **segment tree** (E-maxx.ru, n.d.) is a data structure that efficiently supports range sum queries on an array of numbers. Alternatively, a **Fenwick tree** (Fenwick, 1994), also known as a binary indexed tree, can be used. In either case, the internal implementation details of these data structures can be treated as black boxes.

The segment tree operates on an array with N numbers and supports three primary operations:

1. **Add a value X to the number at the position i:** This operation updates a specific array element by incrementing its current value by X.
2. **Calculate the sum of numbers in the interval $[L, R]$:** This operation calculates the sum of all numbers between indices L and R, inclusive.
3. **Find the smallest position K where the cumulative sum of numbers in the range $[1, K]$ exceeds a given value X:** This operation finds the smallest index K

such that the cumulative sum from the start of the array to index K exceeds the specified threshold X.

Segment trees efficiently perform operations in logarithmic time, making them suitable for scenarios involving frequent updates and range queries. Returning to the original problem, the p-th percentile calculation algorithm follows this process:

Segment Tree initialization: Initialize a segment tree with a size of 1000, where each position in the range 1 to N=1000 corresponds to the frequency of resource usage values, ranging from 0% to 100%, in increments of 0.1%.

The algorithm adjusts the time window by shifting both its start and end positions forward by one step. When the time window moves, the algorithm updates the frequency counts by incrementing the count for the resource usage value at the new position and decrementing the count at the position being removed.

To calculate the p-th percentile, the algorithm offers two approaches:

1. **Using a query to find the position:** The algorithm can utilize the third type of query to directly locate position k in the data structure.
2. **Binary search on the value X:** Alternatively, a binary search can be performed on the value X, leveraging the second query that calculates the sum for the interval $[1, X]$. This approach allows the algorithm to determine the percentile by searching for the value at which the cumulative frequency exceeds the specified percentile threshold.

Alternative approach

An alternative approach that avoids complex data structures, such as priority queues and segment trees, involves reducing the frequency of p-th percentile calculations by performing them at predefined intervals (e.g., every 6 hours). This approach utilizes a deque to store data within the specified time window. When a p-th percentile calculation is required, the values in the deque are copied into a new vector, sorted, and the p-th percentile is calculated.

In contrast to the first two approaches, the alternative approach recalculates the p-th percentile after a fixed interval, using the complete resource usage history. If recalculations are frequent, this approach becomes impractical.

5.10 Summary

This chapter presented a cloud simulator that replicates the behavior of real cloud computing environments. The simulator allows users to study several workloads, resource allocation strategies, and overcommitment policies without having the cost and risk of running experiments on actual cloud infrastructure.

In the cloud simulation space, several simulators exist and are widely used in research, such as CloudSim and CloudSimPlus. Limitations in these frameworks motivated the

development of Clovers, a new simulator written in Rust. The use of Rust offers significant benefits, particularly stronger memory safety guarantees than Java.

Clovers is built with a discrete-event simulation engine that processes events including the addition and removal of containers and machines, manages overload conditions, supports several overcommitment policies, and implements scheduling algorithms. Clovers also provides a resource usage interface for metrics for analysis.

To evaluate the outcomes of different policies, the simulator tracks SLA violations by measuring the frequency and magnitude of time periods in which containers' resource usage exceeds their allocated resources. The simulator also monitors the total number of machines utilized, facilitating analyses across different policies.

The final section of this chapter discussed methods for calculating the p-th percentile, which is important for implementing container-centric overcommitment, by presenting three possible approaches: a priority queue, a segment tree, or periodic recalculations. It is important to note that the simulator is not designed to perfectly replicate real-world uncertainties.

Chapter 6

Data collection

To perform overcommitment experiments, trace data from CSPs are required. These trace data contain detailed information about the internal environments of the CSPs, including container and machine resource usage across CPU and memory. Since the infrastructure of each CSP differs, the types of information provided also change. The information about each of these CSPs must be completely understood and analyzed to extract what is useful in the experiment.

6.1 Trace data source

The open-source trace data used in the research come from Google Borg, Microsoft Azure, and Alibaba Cloud. The trace data provided by these CSPs are only partially available due to privacy concerns. Full disclosure of the data could potentially be exploited, leading to issues such as competitive risks and the leakage of users' private information (Reiss, Wilkes, *et al.*, 2012). A similar issue occurred with other datasets, such as the Netflix Prize dataset, in which researchers demonstrated that it was possible to extract users' information even from anonymized datasets (Narayanan and Shmatikov, 2007).

Each CSP uses a unique data format and type of information in its trace data. The following sections explore the details provided by each CSP and also suggest an alternative method for generating trace data using a synthetic workload.

6.1.1 Google Borg

Google Borg Cluster is a cluster management system that handles hundreds of thousands of jobs from thousands of applications across multiple clusters spread worldwide, each comprising up to tens of thousands of machines. The datasets generated by these clusters provide information at a terabyte scale.

The Borg cluster is used exclusively for internal Google products and services. As the predecessor to Kubernetes, Borg introduced key ideas and features that were later integrated into Kubernetes. More details about the Borg system are available in its original paper (Verma *et al.*, 2015).

The Google trace provides detailed information about containers and machines running in a heterogeneous environment, where resource capacities change significantly. CPU and memory allocation for containers, as well as their time series, are normalized between 0 and 1. Researchers have used the Google Borg Cluster data to study overcommitment (Bashir *et al.*, 2021).

6.1.2 Azure

Microsoft Azure only provides information about containers but does not include details about the machines where the containers are allocated. As a result, evaluating the CSP's machine utilization effectiveness is impossible.

The dataset includes a subscriber ID that identifies the user associated with the VM. While tasks with the same subscriber ID might suggest they are identical, this is not necessarily the case.

An intriguing aspect of Microsoft Azure Resource Central is its categorization of workloads into human workloads, machine workloads, and others. This classification is highly beneficial for future experiments, as it enables comparisons between batch tasks (machine workloads) and online services (human workloads). Moreover, it allows for a clearer separation into different types of workloads and supports the evaluation of different resource allocation strategies that prioritize one workload type over another.

The cloud trace contains one month of data and includes only the CPU usage time series. Unfortunately, the cloud trace does not provide a time series of memory usage, offering only the average memory usage.

6.1.3 Alibaba Cloud

An interesting aspect of Alibaba Cloud is that its services are divided into two main systems: Sigma for online services and Fuxi for batch workloads.

The trace data cluster-trace-v2017 contains information from a production cluster over a 24-hour period, but the data are available only for 12 consecutive hours. It includes information from approximately 1.3k machines running both online services and batch jobs. However, due to a failure in the trace collector, there is a period during which data are missing.

The limited time range makes it challenging to understand the tasks fully and derive effective overcommitment policies. Additionally, the dataset requires further analysis to determine whether the lack of data significantly impacts the ability to obtain meaningful results for overcommitment.

The cluster-trace-v2018 dataset contains information from 4,000 machines over an 8-day period, providing a larger dataset compared to cluster-trace-v2017. This increased data volume enhances the dataset's interval, enabling more effective simulations that utilize container information.

6.1.4 Synthesized workload

Synthesized workloads provide a different way to create trace data, allowing the testing of how containers perform when overloaded and helping to build realistic simulations. These processes can be time-consuming to reproduce and may require substantial processing power to execute.

SDCBench provides a comprehensive list of benchmarks and evaluates various workload benchmarks available for simulation (Yang *et al.*, 2022). TailBench (Kasture and Sanchez, 2016), designed for testing latency-critical applications, will be used to analyze the relationship between CPU usage and latency. Additionally, the DataDwarf (Gao *et al.*, 2018), designed for studying big data and AI workloads, may also be utilized.

An important feature of synthesized workloads is the ability to define the type of workload in each container. A possible downside of synthesized workloads is that if the benchmark is not updated for a long time, any major changes in the required internal packages will require updates to the benchmark's internal code.

Tools like **RRDtool** and **crontab** are not utilized in the current implementation. They may be integrated later to gather information from synthesized traces. **RRDtool** is a software for managing time series data, while **crontab** is a Linux utility for scheduling scripts at specified intervals. These tools could enable continuous monitoring of CPU and memory consumption over time.

6.1.5 Comparison

A notable difference is that Microsoft does not provide memory time series data. Memory usage is generally more stable than CPU usage, which often experiences frequent spikes. This stability allows us to approximate memory usage based on available data.

The key differences between the datasets are summarized in the table below.

Dataset	Time	Container Alloc[1]	CPU TS[1]	Memory TS[1]	Container Name
Google (2011)	90 days	✓	✓	✓	✓
Google (2017)	90 days	✓	✓	✓	✓
Microsoft (2017)	30 days	✓	✓		
Microsoft (2019)	30 days	✓	✓		
Alibaba (2017)[2]	0.5 days	✓	✓[3]	✓[3]	
Alibaba (2018)	8 days	✓	✓[3]	✓[3]	

Table 6.1: *Dataset comparison - Container*

The dataset also includes additional information, such as machine resource usage data and the execution order of containers, which are represented as a directed acyclic graph (DAG).

[1] "Alloc" is an abbreviation for allocation, while "TS" is an abbreviation for time series.

[2] There were issues during trace data collection that resulted in missing container usage data.

[3] The usage data for batch tasks is not available.

Dataset	Machine	DAG
Google (2011)	✓	
Google (2017)	✓	✓
Microsoft (2017)		
Microsoft (2019)		
Alibaba (2017)[2]	✓	
Alibaba (2018)	✓	✓

Table 6.2: *Dataset comparison - Extra*

6.2 Container classification

This section looks at RQ3 from Chapter 1, which investigates how containers can be categorized into classes to optimize the effectiveness of overcommitment policies. The thesis did not provide a solution to the problem of segmenting and clustering workloads. Google Autopilot emerged as the only approach that showed potential for further research. Moreover, other methods lack sufficient container-specific information, making it difficult to derive meaningful results.

Segmentation involves dividing data into groups based on similar characteristics. Grouping containers by similar behaviors can help the system predict and generate custom overcommitment policies for each group. One possible approach is to analyze container data in greater depth by collecting detailed information, such as the operating system and container-specific attributes, and then generating decision trees. However, the data are not publicly available (Cortez *et al.*, 2017).

Clustering is a technique for grouping data based on similarity. Clustering trace time series is particularly challenging because time series from the same container class can display distinct shapes (Kedia and Makagonov, 2018). Even within the same workload, time series patterns can change significantly. As a result, algorithms such as Dynamic Time Warping (DTW) or K-shapes (Paparrizos and Gravano, 2016) may not perform well in these scenarios.

When a container trace exhibits a periodic pattern over time, this behavior can be exploited. An algorithm based on the Fast Fourier Transform (FFT) can identify the positions in the time series that are most likely to correspond to period repetitions. Microsoft Resource Central leverages this information to distinguish between human and machine containers. In this context, a human container is defined as a container that interacts with a human and can be affected by latency, while a machine container is a container used solely for processing information. Although FFT can find the approximate period of repetition, distinguishing between human and machine containers remains challenging due to the complexity of interpreting the trace data in some cases (Cortez *et al.*, 2017).

6.3 Implementation

This section describes the data collection process and its implementation. The source code is available at https://www.gitlab.com/cloudsimrs/trace-data.

6.3.1 Tools

To accelerate data processing speed through parallelism, the data collection process takes advantage of two primary tools: Polars and Rayon.

Polars

Similar to Pandas, Polars is a Rust-based high-performance data manipulation library that efficiently processes large datasets through multi-threading. It offers APIs for both Python and Rust, often providing improved speed and lower memory usage for large-scale data processing. The developed code uses Rust's native DataFrame functionality directly, ensuring optimal performance.

As a relatively new and rapidly evolving library, Polars experiences frequent syntax changes, such as updates to the 'partition_by' function and DataFrame read formats, which have required modifications to the code throughout the development process.

Rayon

Rayon is a Rust library for parallel processing, designed to enable concurrent data processing by efficiently distributing workloads across multiple CPU cores.

6.3.2 Data extraction process

The process begins with downloading the trace dataset. After the download, SHA256 checksums are applied to the files to verify their consistency. Once the integrity of the files is confirmed, the dataset is processed to extract important information.

Information about containers and machines is collected, including details on CPU and memory allocation and time series data on CPU and memory usage. Additionally, the relationship between containers and machines, along with information about the container's owner, is available.

An optimization in data reading is batch processing. Instead of reading each file individually, multiple files can be merged into a single file for processing information in bulk. This approach minimizes the overhead associated with file operations and improves read performance.

The container and machine IDs must be parsed and converted to integers to optimize query performance, as querying by IDs with non-integer formats is inefficient. A hash table should be created to map these values to their corresponding integer representations, and the container and machine IDs need to be converted to integers.

After segmenting the container and machine information, the experiment generation process begins. To ensure consistency across runs of the same experiment, a fixed seed

with ChaCha8 is used. This method ensures that the generation of machine and container IDs remains consistent between runs with the same seed.

6.3.3 Alibaba Fuxi/Sigma

The trace data from Alibaba Cloud can be accessed at https://github.com/alibaba/clusterdata.

A problem was identified in the Alibaba Fuxi/Sigma dataset from 2017. The dataset contains missing information during the start timestamp range between 60K and 89K. Specifically, 85% of entries in the batch_instance.csv file are missing the 'task_id' and/or 'job_id'. Therefore, the dataset from 2017 is not used due to this missing information.

The information about failure domains in the Alibaba Cloud dataset is not well explained, making it difficult to determine what should be excluded. As a result, failure domains are not addressed, and all available data are included. Additionally, the dataset does not provide time series data on CPU and memory usage for batch tasks, so resource usage over time can only be collected for container data.

6.3.4 Microsoft Azure

The dataset can be found in the GitHub repository at https://github.com/Azure/AzurePublicDataset/tree/master.

One key insight from the Microsoft Azure Cluster dataset is the identification of workload types, which helps distinguish between latency-sensitive and non-latency-sensitive workloads. However, a limitation of the dataset is that it does not include machine and container memory time series.

Since the container ID and user ID are stored as long hashed strings, search operations become inefficient due to the need for full string comparison. Converting these IDs to integers before searching can significantly improve efficiency.

6.3.5 Google Borg

Information about the Google Borg trace is available in the GitHub repository at https://github.com/google/cluster-data/blob/master/ClusterData2019.md.

The repository provides details about the datasets, which include both container-related and machine-related information. Cloud trace data can be accessed from two sources. For this experiment, the dataset was retrieved from Google Cloud Storage (GCS) as compressed JSON files, while the data are also available directly via Google BigQuery, which offers the benefit of allowing the user to select a specific time interval.

The download process uses *gsutil*, a tool designed for managing resources on the Google Cloud Platform (GCP) interface. Installation instructions for *gsutil* can be found at https://cloud.google.com/storage/docs/gsutil_install?hl=pt-br.

The following script demonstrates how to use *gsutil* to download the compressed JSON files:

```
gsutil cp gs://clusterdata_2019_a/instance_usage-*.json.gz /Volumes/
    Expansion/data/google/instance_usage/
```

The cluster data were downloaded from one of the regional cells (e.g., A — New York) and stored on an external storage device (Seagate HDD 6 TB). Downloading 1 TB of data takes approximately 3 to 4 days to complete, depending on network speed. After downloading, an extractor separates the cluster data by machine ID for each JSON shard using *Polars* and the *partition_by* function, converting the data into a Parquet file. The processed data are then handled on the university server, Brucutu VI, which requires about one week to complete the task. To transfer data between Brucutu VI and the external hard drive, Google Cloud Storage (GCS) is used as an intermediary storage solution.

Brucutu VI is one of the processing servers at IME-USP, with its hardware specifications detailed in Chapter 7. The server operates under specific rules: only half of the available threads may be utilized, and tasks that violate this rule are terminated by the server administrator. Due to the large size of the dataset, the data extraction algorithm is divided into two phases. The first phase, the partitioning phase, partitions the data by machine ID, and the second phase merges the data by machine ID.

Partitioning phase

The first phase is the partitioning phase. Initially, the cluster data are unstructured and not separated. During this phase, the data are organized and partitioned based on the machine ID (machine) and the task ID (container).

For each JSON shard, the extractor generates a list of machine IDs extracted from the shard. This list is used to identify inconsistencies in the data. Each machine ID is associated with a list of JSON shard IDs, and the JSON shard files corresponding to each machine ID are joined together. To enable rapid queries, the extractor also creates a reverse mapping for each machine ID, linking it to the JSON shard IDs used to generate its file.

Subsequently, for each machine ID, the extractor partitions the data into lists of containers. These tasks are then separated and processed in parallel using Rayon's scoped threads.

Merging phase

The merging phase separates the data based on machine IDs. This process generates time series information for each machine and produces a list of containers, preparing the data for simulation. Before running the simulation, the container information for each machine must be sorted. The data itself isn't merged but remains separated, as merging takes time and increases the size of the data to be processed.

The extracted information includes time series data for running processes. The data are segmented into 5-minute intervals, and any gaps between data points are filled using the most recent value.

6.4 Summary

The chapter presented datasets from different CSPs, such as Google Borg, Microsoft Azure, and Alibaba Cloud. Each dataset has specific characteristics and limitations regarding format and information. The Google Borg dataset provides only normalized CPU and memory usage data. The Microsoft Azure dataset categorizes containers into human and machine workloads, but lacks information about the memory usage time series. The Alibaba Cloud dataset is limited by restricted time intervals and incomplete data due to collection issues, and the dataset does not have time series information about batch tasks, only online services. As an alternative to datasets, synthetic workloads can be used. Benchmarks such as SDBBench, TailBench, and DataDwarf can generate realistic workload traces and execute in real environments under overload scenarios. Due to time constraints, only TailBench was tested.

Several container classification approaches were analyzed, but optimizing overcommitment policies using these container grouping ideas remains a challenging and open problem. There is no publicly available dataset about containers to enable effective clustering, and time series data clustering algorithms do not perform well for workloads when there is variability within the same class. A periodicity-based approach was also explored, but research indicates that even this approach struggles to differentiate between human workloads and machine workloads. The only option seems to be creating a model like Google Autopilot.

The final section of this chapter includes implementation details about data extraction. Polars and Rayon are the primary tools used in the extraction process. Polars is used for all trace data, while Rayon is used specifically for the Google Borg dataset. The process of extracting data involves checking data consistency, using batch processing to speed up reading, applying a fixed hash to generate the data for reproducible simulation runs, and managing some special optimizations for each CSP.

Chapter 7

Experiments

The experiments for overcommitment were conducted with the hardware specification outlined in **Table 7.1**. Two machines were used: **Machine A**, a personal computer (PC), and **Machine B**, a processor server from IME/USP faculty (Brucutu VI). During data extraction, the data was distributed between Brutucu VI and the PC for data processing. The trace dataset and the selected information were stored on an external hard drive (Seagate HDD 6 TB). In the experiment, Brutucu VI was used for large-scale testing, while the PC was used for small-scale testing.

(A) *PC*

CPU	AMD Ryzen 5 5600G
Core	6 cores
Thread	12 threads
RAM	32 GB
GPU	AMD Radeon RX Vega 7
OS	Windows 11

(B) *Brucutu VI*

CPU	2 x Intel Xeon Gold 6148
Core	20 cores
Thread	40 threads
RAM	512 GB
GPU	Nvidia Tesla V100
OS	Debian/Linux

Table 7.1: *Hardware specification*

7.1 CPU usage effects on latency

This experiment aims to explore the relationship between CPU usage and latency in a real cloud environment from a latency-critical application perspective. The workloads used in this experiment are derived from TailBench (Kasture and Sanchez, 2016). While the referenced research primarily examined the relationship between the number of requests and latency, this experiment analyzes the relationship among the number of requests, latency, and CPU usage.

TailBench is a benchmark designed to evaluate latency performance for latency-sensitive workloads. The benchmark includes configurations for in-memory, loopback, and network connections to measure latency. For this evaluation, only the in-memory configurations are presented, as the other configurations produce similar results.

Table 7.2 presents the types of workloads in TailBench. Each workload is designed to simulate a real-world application, allowing a detailed analysis of latency behavior under controlled CPU usage conditions.

Name	Workload type	Language
img-dnn	Image recognition	C++
masstree	Key-value store	C++
moses	Statistical machine translation	C++
shore	OLTP database (optimized for disk/ssd)	C++
silo	OLTP database (in-memory)	C++
specjbb	Java middleware	Java
sphinx	Speech recognition	C++
xapian	Online search	C++

Table 7.2: *TailBench workloads*

The number of requests is gradually increased until CPU usage slightly exceeds 90%, a threshold referenced in the literature as the overload limit. Another parameter for selecting this threshold is identifying the exact point at which latency begins to grow too quickly.

Experimental analysis indicates that certain applications, such as img-dnn, have an overload threshold near 70%. Beyond this point, the system becomes unstable, signaling an overloaded state in which the CPU becomes unreliable and latency increases rapidly.

The majority of workloads, including img-dnn, masstree, moses, and xapian, exhibit a steady increase in CPU usage as the number of requests grows. In contrast, workloads such as shore, silo, and specjbb experience an initial spike in CPU usage before stabilizing at normal levels. This early surge often causes the recorded maximum CPU usage to reach 100%. Similarly, the sphinx workload processes voice data in intermittent bursts, resulting in periodic spikes where CPU usage hits 100%. Because the information is aggregated, these peaks also contribute to the recorded 100% CPU usage.

Figure 7.1 presents a comparison between CPU usage and latency across the TailBench workloads. In each subfigure, the red line indicates CPU usage, while the blue line represents latency. The left y-axis displays the latency values, and the right y-axis displays the CPU usage values.

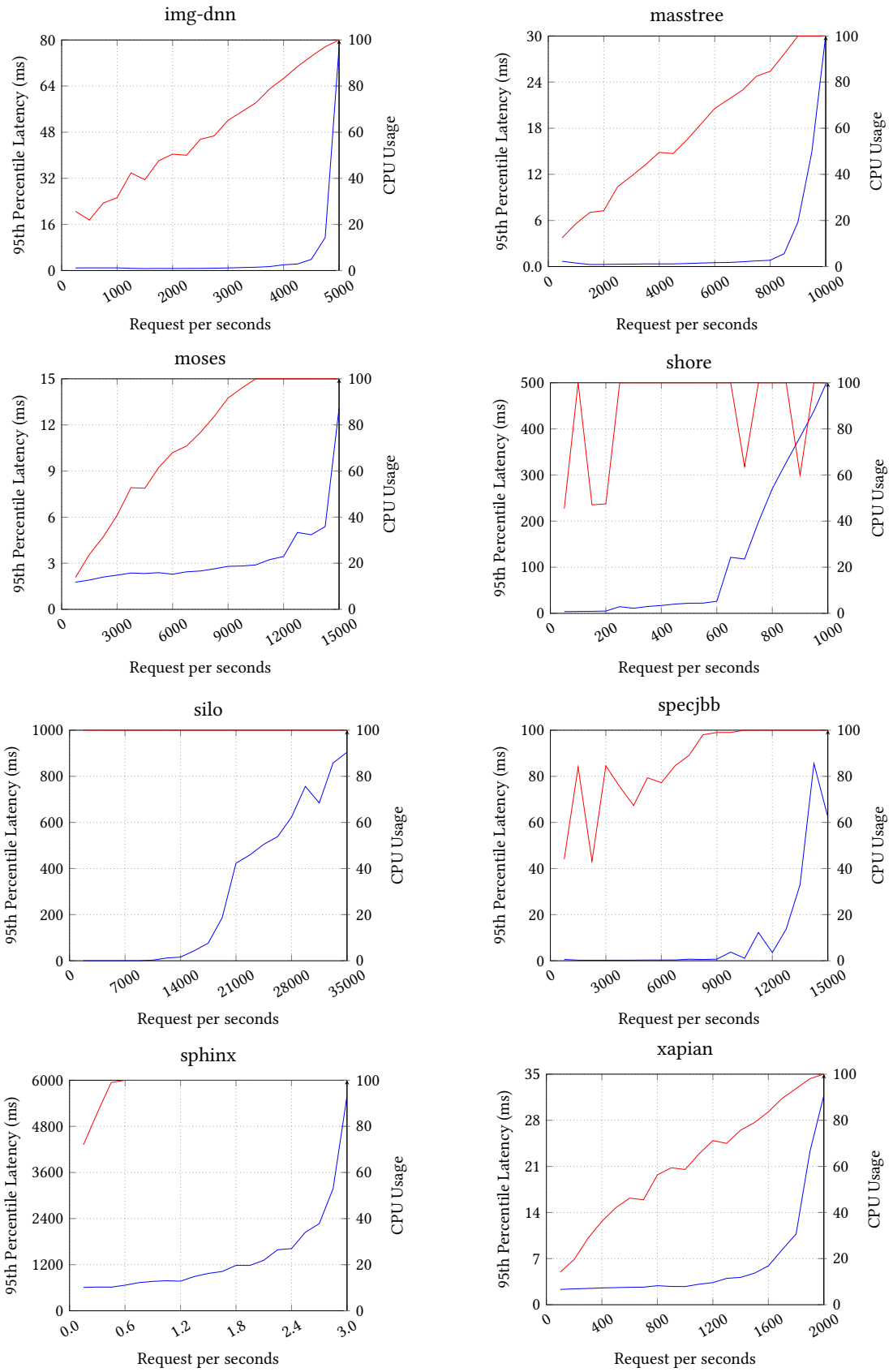

Figure 7.1: *Comparison between CPU usage and latency*

Latency is influenced by the accumulation of computational requests. When an application handles a computationally intensive request, subsequent requests can quickly queue up. As processing each request takes time, this leads to increased overall latency. The faster the server accumulates such requests, the higher the resulting latency.

Another important consideration for Java applications like specjbb is the possible impact of garbage collection, although garbage collection was not triggered during the experiment.

The variability in CPU usage of latency-sensitive workloads across containers highlights the importance of understanding container-specific characteristics. Such insights are crucial for designing effective container-centric dynamic overcommitment policies.

7.2 Evaluation of resource underutilization

To effectively evaluate resource waste, it is essential to define metrics that provide meaningful insights into resource utilization. One fundamental metric is the resource waste ratio, calculated as the ratio of wasted resources to total resource capacity. This ratio measures the proportion of unused resources relative to the total available capacity, offering a clear perspective on resource allocation efficiency within the CSP environment.

This approach analyzes trace data to identify patterns in resource usage inside CSP. Different types of workloads have distinct resource underutilization, and the defined metric contributes to understanding the resource efficiency of CSP.

While it is preferable to evaluate each machine individually to determine total resource utilization, some datasets lack details on container-to-machine allocation, making direct machine allocation analysis impossible. Despite this limitation, machine usage can still be inferred by analyzing container usage and the resource limits assigned to each container.

The analysis could be more interesting if the dataset included additional information, such as the number of users accessing the service or the volume of queries per hour. This data would provide insights into user behavior and allow for the identification of daily access trends. For example, in low-traffic environments, like personal sites, this can demonstrate how overcommitment strategies can be effectively implemented. Since this information is not available in the current dataset, the analysis must focus on the CPU and memory usage ratios to examine resource underutilization.

For the analysis of resource underutilization in CSP, a seven-day interval is selected for the Alibaba Trace data. For each day of the week, data points on used and allocated resources are collected and organized. The x-axis is divided by hour and categorized by container or machine when machine-level data are available. The Alibaba Trace data do not provide an exact start time. To prevent initialization issues in the Alibaba Trace data, any data that have a timestamp below 600 are excluded, and the analysis begins at timestamp 600.

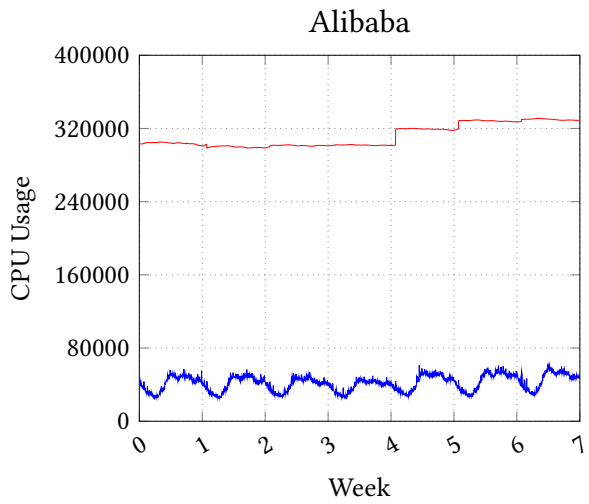

Figure 7.2: *Alibaba - CPU*

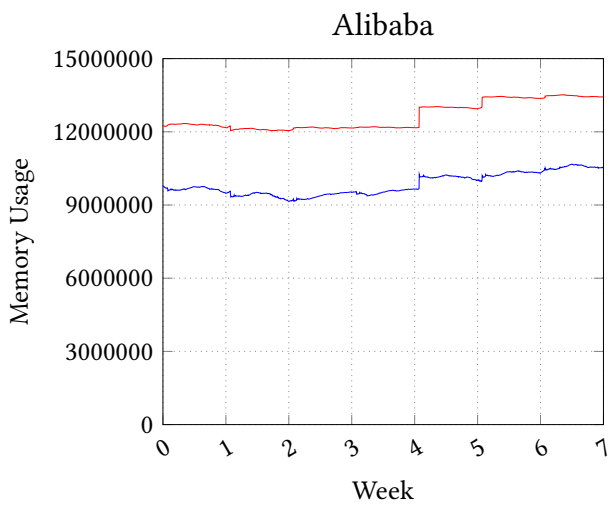

Figure 7.3: *Alibaba - Memory*

Alibaba resource data show a skewed distribution between CPU and memory allocation, with a greater utilization of memory than CPU. This imbalance may significantly influence container placement on machines. Figure 7.2 also indicates that CPU utilization remains low, showing that these CPU resources could potentially be better utilized.

7.3 Comparison between overcommitment policies

The basic overcommitment strategies are tested using the Microsoft Azure trace dataset (2019). The experiment configuration includes a first-fit scheduling algorithm, sets an overload handler to "no-limit", and uses an evaluator to generate the number of machines used, the violation size, and the violation count. The overcommitment policies that are compared are machine-centric and container-centric static approaches to CPU resources. When a container cannot be placed on any machine, a new machine is added to the cluster. Each machine has 50 units of CPU and 200 units of memory.

The simulation is executed 100 times, each run with 100 containers selected at random, and the average of the results from the evaluator is calculated. For the reproducibility of the experiment, each run has the same seed for ChaCha8, and this random number generator seed ensures the consistency among the different runs.

7.3.1 Machine-centric overcommitment

In the machine-centric approach, the experiment evaluates a static overcommitment policy where the overcommitment rate is set at 100%, 110%, 120%, and 130%. By observing how each rate affects resource usage and violation counts, the study aims to characterize the effectiveness of static overcommitment rates at the machine level.

7.3.2 Container-centric overcommitment

Static

In this approach, the overcommitment rate is determined by the p-th percentile of container resource usage. Specifically, the experiment evaluates policies using the 90th percentile, the 95th percentile, the maximum usage values, and a baseline with no overcommitment.

A key hypothesis is that lowering the percentile, such as from the 95th to the 90th, will result in a higher violation rate as the resources become more compactly allocated. The objective is to identify the lowest percentile threshold at which the violation rate remains negligible, ideally zero. To evaluate this, four distinct policies are compared:

- Overcommitment based on the 90th percentile.
- Overcommitment based on the 95th percentile.
- Overcommitment based on the maximum usage value.
- No overcommitment.

Table 7.3 presents the results of the overcommitment experiment of CPU resources, indicating that the overcommitment policy works in theory.

(A) *Machine-centric*

Resource Size	Machine	Count	Size
100%	1.98	0.00	0.000000
110%	1.90	0.00	0.000000
120%	1.81	0.00	0.000000
130%	1.78	0.01	0.010085

(B) *Container-centric static*

Resource Size	Machine	Count	Size
Baseline	1.98	0.00	0.000000
Max	1.80	0.00	0.000000
95th	1.76	0.00	0.000000
90th[1]	1.76	0.00	0.000000

Table 7.3: *Result from the overcommitment experiments*

This experiment evaluates the overcommitment strategy by comparing the machine-centric approach with the container-centric static approach. A comparison between the baseline and 90th-percentile scenarios shows that the average number of machines decreases from 1.98 to 1.76, which corresponds to an 11% reduction in the number of machines. This improvement indicates more efficient machine packing. Another observation is that the machine-centric approach is more prone to violations than the container-centric static approach.

7.4 Summary

This chapter constitutes the experimental component of the thesis and presents three experiments designed to understand the cloud environment where resources are overcommitted. When using a simulator, it is important to understand the accuracy of the model and its effects compared to the real environment. Such accuracy evaluation is not possible in this study due to the use of historical trace data, which cannot be re-executed for validation.

The first experiment investigated the relationship between CPU usage and latency. To examine this relationship, the TailBench benchmark was evaluated using a predefined set of workloads. The experiment found that when CPU capacity is exceeded and unable to handle incoming queries, latency increases exponentially. The degree of latency differs for each workload, suggesting that each container should be analyzed individually when predicting latency.

The second experiment assessed resource utilization levels in CSP workloads by visualizing usage patterns for individual resources. This analysis aimed to provide a clear understanding of how significantly the system is underutilized and to identify potential benefits and issues that overcommitment can bring to CSP.

[1] Violations begin to occur at the 80th percentile.

The third experiment evaluates basic resource overcommitment techniques. Results indicate that applying overcommitment reduces the total number of machines by approximately 11% compared to scenarios without overcommitment. This reduction represents a significant theoretical outcome.

Chapter 8

Conclusion

The results presented in this thesis are partial because the scope of the project was too extensive for one person to manage. Implementing each component and extracting information from time series data on the order of gigabytes to terabytes is time-consuming. Another issue is the lack of efficient classification and clustering tools in the literature designed for time series datasets of cloud traces and the absence of detailed container information needed to categorize workloads. During the experiment design, it was assumed that workloads could be divided into just two categories, batch tasks and online services, but this simplification does not capture the complexity of real-world scenarios encountered in CSP environments.

Different cloud trace datasets were examined and found appropriate for testing overcommitment policies. The Google Borg dataset is particularly useful for evaluating how containers are allocated to machines, while the Microsoft Azure dataset is better suited for analyzing overcommitment between batch tasks and online services. These datasets, including the Alibaba dataset, provide valuable insights into CSP's resource utilization.

A simulator was developed to allow users to test data from cloud trace datasets and try different overcommitment policies before deploying the workloads in a production environment. The simulator also includes tools for evaluating overload mechanisms, scheduling methods, and performance metrics. Additionally, it supports the allocation of machines and containers in multiple sizes, enabling users to simulate different heterogeneous resource configurations. In the future, time-bound overcommitment is going to be implemented, but it currently remains incomplete as it needs to develop a better resource history for tracking execution times, and also finish the overload mechanism.

Overcommitment has proven to be effective in optimizing resource usage in the cloud. Users often overestimate their resource requirements, resulting in unused resources that can be reallocated to other containers. Leveraging these unused resources usually presents minimal challenges, especially for applications that do not have strict latency or completion time requirements.

The effectiveness of memory and CPU overcommitment depends largely on the type of workload. Overcommitting resources may not be ideal for CPU-intensive containers that experience frequent spikes or for latency-sensitive workloads, but it can be very

beneficial for containers that consistently use low resources and experience only occasional spikes.

In a Kubernetes cluster, unused resources should be resized to match the containers' actual needs. Fine-tuning resource limits helps reduce costs and align the infrastructure with current demand levels (Google Cloud n.d.).

Reducing the QoS for lower-priority containers during overcommitment can lead to container termination, slowdowns, or unresponsiveness. Determining an optimal overcommitment policy is challenging because it requires assessing each container to evaluate the overall impact on system performance.

If the cluster is not overcommitted, but the cluster manager wants to try out overcommitting machines or containers, a straightforward strategy is to start with a fixed overcommitment rate and monitor system performance. If operations remain smooth, gradually increase the rate, but if issues emerge, reduce the rate. Continue this iterative process until an optimal overcommitment rate is reached.

To implement overcommitment on a per-container basis, a dynamic strategy can be adopted. For each container, the overcommitment rate is determined based on the p-th percentile of its workload type. The violation rate is then monitored to ensure that performance remains within predefined acceptable limits.

A potential direction for implementing automatic overcommitment is to adopt concepts from Google Autopilot. This approach creates a resource allocation model based on factors such as underrun, overrun, safe margins, and decay rates. It applies reinforcement learning-like techniques to find the most suitable model for each container's resource usage. By directly determining the optimal allocation, this approach avoids clustering algorithms.

While the clustering-based approach relies on clustering features, the lack of detailed container information in publicly available datasets makes replicating the strategy used by Microsoft Resource Central infeasible.

8.1 Future direction

Throughout the development of this work, several promising ideas have emerged that can extend the understanding of overcommitment and provide opportunities to explore certain topics in greater depth. Additionally, the existing concepts presented in this thesis could also be further refined and enhanced, building on the progress made so far.

8.1.1 Comparison of resource underutilization among other CSPs

Due to time constraints, as data processing for datasets ranging from 100GB to 1TB is slow, resource underutilization data for the Azure and Borg trace datasets were not included, as accurate information on resource usage and allocation could not be generated.

The information could provide a contrast between different CSPs' resource underutilization.

8.1.2 Comparison of different time windows in resource history

Cloud environments are highly dynamic, with tasks constantly changing. Relying on long-term historical data for calculations may introduce outdated information, potentially resulting in suboptimal overcommitment decisions for specific workload types.

8.1.3 Comparison among scheduling algorithms

Several scheduling algorithms can be employed when allocating containers to machines, with the goal of efficiently packing containers to minimize the number of machines required. Research indicates that the dot-product algorithm performs slightly better than others, although there are scenarios where alternative algorithms may outperform it (Panigrahy *et al.*, 2011). There is also a dataset specifically designed for vector bin packing, such as the Microsoft Bin-packing dataset, that could be used to test and validate these algorithms.

8.1.4 Comparison among overload handling

The overload handling experiment in the Google Autopilot paper (Rzadca *et al.*, 2020) evaluated OOM under soft-limit and hard-limit memory mechanisms. The results showed significant differences in outcomes related to out-of-memory (OOM) events. This overload handler could be extended to include live migration for handling overload in other types of overload scenarios.

8.1.5 Automation of overcommitment policies

Developing built-in automated overcommitment policies for Kubernetes clusters could eliminate the need for administrators to manually configure node resource limits, allowing them to focus on other tasks. A similar approach is employed in Google Autopilot, which tests several parameters, such as overrun limit, underrun limit, safety margins, and decay rates, combined with a model evaluation technique inspired by reinforcement learning.

8.1.6 Clustering-based overcommitment policies

Clustering time series data to associate patterns with container behavior presents significant challenges, largely due to the limited details available in trace data. Although patterns in the time series can be identified, effectively clustering these patterns to analyze overcommitment has proven difficult. Several clustering algorithms have been evaluated, but the literature indicates low effectiveness, primarily due to the dataset variability within the same workload types.

A key limitation of public datasets is the lack of detailed container information, such as machine and operating system configurations, as well as container-level information. The lack of data restricts accurate analysis and modeling of container behavior.

8.1.7 Machine learning in overcommitment policies

Machine learning can play a crucial role in selecting among various overcommitment policies or determining appropriate overcommitment rates. The following three examples from the literature illustrate how machine learning techniques can be applied to overcommitment:

Microsoft Resource Central extracts information from containers to generate predictive models using algorithms such as Random Forest and Extreme Gradient Boosting (XGBoost). These models infer container utilization brackets, and based on the inferred utilization, an overcommitment policy rate is determined (Cortez *et al.*, 2017). However, the absence of container information, as noted earlier regarding the limitations of the public dataset, makes this method non-reproducible.

Google Autopilot leverages machine learning techniques to develop a model that fits the container. The system incorporates factors such as underrun, overrun, safety net, and decay rate to determine which model is most appropriate to apply. Autopilot adopts a model similar to reinforcement learning (RL), selecting the model that achieves the highest reward based on these criteria (Rzadca *et al.*, 2020).

Unlike Google Autopilot, chance-constrained multi-agent reinforcement learning (C2MARL) uses a specific framework to determine overcommitment rates among multiple agents (Sheng *et al.*, 2023). This framework employs relaxation techniques to find an approximate solution through linear programming constraints, thereby generating multi-agent coordination.

Reinforcement learning is challenging due to the complexity of designing effective reward and penalty systems for agent actions. Another challenge is balancing exploration and exploitation to ensure that the agent consistently learns optimal actions, which complicates policy development using reinforcement learning.

References

[Andreoli *et al.* 2024] Remo Andreoli, Jie Zhao, Tommaso Cucinotta, and Rajkumar Buyya. *CloudSim 7G: An Integrated Toolkit for Modeling and Simulation of Future Generation Cloud Computing Environments*. 2024. arXiv: 2408.13386 [cs.DC]. URL: https://arxiv.org/abs/2408.13386 (cit. on p. 25).

[Baset *et al.* 2012] Salman A. Baset, Long Wang, and Chunqiang Tang. "Towards an Understanding of Oversubscription in Cloud". In: *2nd USENIX Workshop on Hot Topics in Management of Internet, Cloud, and Enterprise Networks and Services (Hot-ICE 12)*. San Jose, CA: USENIX Association, Apr. 2012. URL: https://www.usenix.org/conference/hot-ice12/workshop-program/presentation/baset (cit. on pp. 5, 15).

[Bashir *et al.* 2021] Noman Bashir *et al.* "Take It to the Limit: Peak Prediction-Driven Resource Overcommitment in Datacenters". In: *Proceedings of the Sixteenth European Conference on Computer Systems*. EuroSys '21. Online Event, United Kingdom: Association for Computing Machinery, 2021, pp. 556–573. ISBN: 9781450383349. DOI: 10.1145/3447786.3456259. URL: https://doi.org/10.1145/3447786.3456259 (cit. on pp. 12, 18, 38).

[Bittencourt *et al.* 2018] Luiz F. Bittencourt, Alfredo Goldman, Edmundo R.M. Madeira, Nelson L.S. da Fonseca, and Rizos Sakellariou. "Scheduling in distributed systems: A cloud computing perspective". *Computer Science Review* 30 (Nov. 2018), pp. 31–54. ISSN: 1574-0137. DOI: 10.1016/j.cosrev.2018.08.002. URL: http://dx.doi.org/10.1016/j.cosrev.2018.08.002 (cit. on p. 28).

[Calheiros 2010] Rodrigo N. Calheiros. "Automated emulation of distributed systems through system management and virtualization". Faculdade de Informáca. PhD thesis. 2010. URL: http://tede2.pucrs.br/tede2/handle/tede/5086 (cit. on pp. 21, 22).

[Carroll *et al.* 2011] Mariana Carroll, Alta van der Merwe, and Paula Kotzé. "Secure cloud computing: benefits, risks and controls". In: *2011 Information Security for South Africa*. 2011, pp. 1–9. DOI: 10.1109/ISSA.2011.6027519 (cit. on p. 5).

[J. Chen *et al.* 2018] Jie Chen *et al.* "Improving Cluster Resource Efficiency with Oversubscription". In: *2018 IEEE 42nd Annual Computer Software and Applications Conference (COMPSAC)*. Vol. 01. 2018, pp. 144–153. DOI: 10.1109/COMPSAC.2018.00027 (cit. on p. 18).

[Cloud Architect Musings 2013] Cloud Architect Musings. *OpenStack Compute for vSphere Admins – Part 4: Overcommitment in Nova Compute*. https://cloudarchitectmusings.com/2013/08/05/openstack-compute-for-vsphere-admins-part-4-overcommitment-in-nova-compute/. Accessed: 2024-10-08. Aug. 2013 (cit. on p. 16).

[Cohen *et al.* 2017] Maxime Cohen, Morteza Zadimoghaddam, Phil Keller, and Vahab Mirrokni. "Overcommitment in Cloud Services – Bin Packing with Chance Constraints". In: *ACM SIGMETRICS / International Conference on Measurement and Modeling of Computer Systems 2017*. 2017 (cit. on pp. 6, 9, 18).

[Cortez *et al.* 2017] Eli Cortez *et al.* "Resource Central: Understanding and Predicting Workloads for Improved Resource Management in Large Cloud Platforms". In: *Proceedings of the 26th Symposium on Operating Systems Principles*. SOSP '17. Shanghai, China: Association for Computing Machinery, 2017, pp. 153–167. isbn: 9781450350853. doi: 10.1145/3132747.3132772. url: https://doi.org/10.1145/3132747.3132772 (cit. on pp. 2, 12, 17, 40, 56).

[Ding 2018] Haiyang Ding. *Alibaba Cluster Trace Program*. https://github.com/alibaba/clusterdata. Accessed: 2025-01-10. 2018 (cit. on p. 2).

[Docker n.d.] Docker. *Docker*. https://www.docker.com. Accessed: 13 March 2025 (cit. on p. 1).

[E-maxx.ru n.d.] E-maxx.ru. *Segment Tree*. Accessed: 2024-03-30. url: https://cp-algorithms.com/data_structures/segment_tree.html (cit. on p. 34).

[Fenwick 1994] Peter M. Fenwick. "A new data structure for cumulative frequency tables". *Software: Practice and Experience* 24 (1994). url: https://api.semanticscholar.org/CorpusID:7519761 (cit. on p. 34).

[Filelis-Papadopoulos *et al.* 2018] Christos K. Filelis-Papadopoulos, George A. Gravvanis, and Panagiotis E. Kyziropoulos. "A framework for simulating large scale cloud infrastructures". *Future Generation Computer Systems* 79 (2018), pp. 703–714. issn: 0167-739X. doi: https://doi.org/10.1016/j.future.2017.06.017. url: https://www.sciencedirect.com/science/article/pii/S0167739X17303230 (cit. on p. 23).

[Forsman *et al.* 2015] Mattias Forsman, Andreas Glad, Lars Lundberg, and Dragos Ilie. "Algorithms for automated live migration of virtual machines". *Journal of Systems and Software* 101 (2015), pp. 110–126. issn: 0164-1212. doi: https://doi.org/10.1016/j.jss.2014.11.044. url: https://www.sciencedirect.com/science/article/pii/S0164121214002751 (cit. on p. 19).

[Gao *et al.* 2018] Wanling Gao *et al.* *Data Dwarfs: A Lens Towards Fully Understanding Big Data and AI Workloads*. 2018. arXiv: 1802.00699 [cs.DC] (cit. on p. 39).

[Google Cloud n.d.] Google Cloud. *Advantages of Cloud Computing*. https://cloud.google.com/learn/advantages-of-cloud-computing. Accessed: 2025-01-10 (cit. on p. 5).

[Google Cloud n.d.] Google Cloud. *Overcommit CPUs on sole-tenant VMs*. https://cloud.google.com/compute/docs/nodes/overcommitting-cpus-sole-tenant-vms. Accessed: 2025-04-05 (cit. on p. 54).

[Grandl *et al.* 2014] Robert Grandl, Ganesh Ananthanarayanan, Srikanth Kandula, Sriram Rao, and Aditya Akella. "Multi-resource packing for cluster schedulers". *SIGCOMM Comput. Commun. Rev.* 44.4 (Aug. 2014), pp. 455–466. issn: 0146-4833. doi: 10.1145/2740070.2626334. url: https://doi.org/10.1145/2740070.2626334 (cit. on p. 29).

[Guo *et al.* 2019] Jing Guo *et al.* "Who Limits the Resource Efficiency of My Datacenter: An Analysis of Alibaba Datacenter Traces". In: *2019 IEEE/ACM 27th International Symposium on Quality of Service (IWQoS)*. 2019, pp. 1–10. doi: 10.1145/3326285.3329074 (cit. on pp. 1, 17).

[Hadary *et al.* 2020] Ori Hadary *et al.* "Protean: VM allocation service at scale". In: *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. 2020, pp. 845–861 (cit. on p. 29).

[Halbuer *et al.* 2024] Alexander Halbuer *et al.* "The new costs of physical memory fragmentation". In: *Proceedings of the 2nd Workshop on Disruptive Memory Systems*. DIMES '24. Austin, TX, USA: Association for Computing Machinery, 2024, pp. 33–40. isbn: 9798400713033. doi: 10.1145/3698783.3699378. url: https://doi.org/10.1145/3698783.3699378 (cit. on p. 10).

[Householder *et al.* 2014] Rachel Householder, Scott Arnold, and Robert Green. "On Cloud-based Oversubscription". *International Journal of Engineering Trends and Technology* 8.8 (Feb. 2014), pp. 425–431. issn: 2231-5381. doi: 10.14445/22315381/ijett-v8p273. url: http://dx.doi.org/10.14445/22315381/IJETT-V8P273 (cit. on p. 10).

[Janus and Rzadca 2017] Pawel Janus and Krzysztof Rzadca. "SLO-aware Colocation of Data Center Tasks Based on Instantaneous Processor Requirements". *CoRR* abs/1709.01384 (2017). arXiv: 1709.01384. url: http://arxiv.org/abs/1709.01384 (cit. on p. 18).

[Jiang *et al.* 2021] Chunmao Jiang, Ling Yang, and Rui Shi. "An energy-aware virtual machine migration strategy based on three-way decisions". *Energy Reports* 7 (2021), pp. 8597–8607. issn: 2352-4847. doi: https://doi.org/10.1016/j.egyr.2021.02.029. url: https://www.sciencedirect.com/science/article/pii/S235248472100127X (cit. on p. 19).

[Kasture and Sanchez 2016] Harshad Kasture and Daniel Sanchez. "Tailbench: a benchmark suite and evaluation methodology for latency-critical applications". In: *2016 IEEE International Symposium on Workload Characterization (IISWC)*. 2016, pp. 1–10. doi: 10.1109/IISWC.2016.7581261 (cit. on pp. 39, 45).

[Kedia and Makagonov 2018] S. Kedia and Sergey Makagonov. "Tuning Apache Spark resource usage for fun and efficiency". Spark + AI Summit 2018. June 4–6, 2018. Spark + AI Summit 2018, San Francisco, California, USA. (Cit. on pp. 13, 40).

[Kim *et al.* 2014] Sangwook Kim, Hwanju Kim, Joonwon Lee, and Jinkyu Jeong. "Group-based memory oversubscription for virtualized clouds". *Journal of Parallel and Distributed Computing* 74.4 (2014), pp. 2241–2256. issn: 0743-7315. doi: https://doi.org/10.1016/j.jpdc.2014.01.001. url: https://www.sciencedirect.com/science/article/pii/S0743731514000033 (cit. on p. 15).

[Kubernetes n.d.] Kubernetes. *Kubernetes*. https://kubernetes.io. Accessed: 13 March 2025 (cit. on p. 1).

[Kulik *et al.* 2023] Ariel Kulik, Matthias Mnich, and Hadas Shachnai. *Improved Approximations for Vector Bin Packing via Iterative Randomized Rounding*. 2023. arXiv: 2205.12828 [cs.DS]. url: https://arxiv.org/abs/2205.12828 (cit. on p. 19).

[Kumbhare *et al.* 2021] Alok Kumbhare *et al.* "Prediction-Based Power Oversubscription in Cloud Platforms". In: *Proceedings of the USENIX Annual Technical Conference (ATC)*. Earlier version published as arXiv:2010.15388, October 2020. USENIX. July 2021. url: https://www.microsoft.com/en-us/research/publication/prediction-based-power-oversubscription-in-cloud-platforms/ (cit. on p. 17).

[Li and Hu 2019] Fengcun Li and Bo Hu. "DeepJS: Job Scheduling Based on Deep Reinforcement Learning in Cloud Data Center". In: *Proceedings of the 4th International Conference on Big Data and Computing*. ICBDC '19. Guangzhou, China: Association for Computing Machinery, 2019, pp. 48–53. isbn: 9781450362788. doi: 10.1145/3335484.3335513. url: https://doi.org/10.1145/3335484.3335513 (cit. on p. 23).

[Linux Kernel Virtual Machine n.d.] Linux Kernel Virtual Machine. *KVM: Kernel-based Virtual Machine*. https://www.linux-kvm.org/page/Main_Page. Accessed: 13 March 2025 (cit. on p. 1).

[Liu 2011] Huan Liu. "A Measurement Study of Server Utilization in Public Clouds". In: *2011 IEEE Ninth International Conference on Dependable, Autonomic and Secure Computing*. 2011, pp. 435–442. doi: 10.1109/DASC.2011.87 (cit. on p. 1).

[Mansi and Swift 2024] Mark Mansi and Michael M. Swift. *Characterizing Physical Memory Fragmentation*. 2024. arXiv: 2401.03523 [cs.OS]. url: https://arxiv.org/abs/2401.03523 (cit. on p. 10).

[Mommessin *et al.* 2025] Clément Mommessin, Thomas Erlebach, and Natalia V. Shakhlevich. "Classification and evaluation of the algorithms for vector bin packing". *Computers and Operations Research* 173 (2025), p. 106860. ISSN: 0305-0548. DOI: https://doi.org/10.1016/j.cor.2024.106860. URL: https://www.sciencedirect.com/science/article/pii/S0305054824003320 (cit. on p. 19).

[Narayanan and Shmatikov 2007] Arvind Narayanan and Vitaly Shmatikov. *How To Break Anonymity of the Netflix Prize Dataset.* 2007. arXiv: cs/0610105 [cs.CR] (cit. on p. 37).

[OpenStack n.d.] OpenStack. *Overcommitting CPU and RAM.* https://docs.openstack.org/arch-design/design-compute/design-compute-overcommit.html. Accessed: 2025-04-03 (cit. on p. 16).

[Panigrahy *et al.* 2011] Rina Panigrahy, Kunal Talwar, Lincoln K. Uyeda, and Udi Wieder. "Heuristics for Vector Bin Packing". In: 2011. URL: https://api.semanticscholar.org/CorpusID:17946270 (cit. on pp. 19, 29, 55).

[Paparrizos and Gravano 2016] John Paparrizos and Luis Gravano. "K-shape: efficient and accurate clustering of time series". *SIGMOD Rec.* 45.1 (June 2016), pp. 69–76. ISSN: 0163-5808. DOI: 10.1145/2949741.2949758. URL: https://doi.org/10.1145/2949741.2949758 (cit. on p. 40).

[Ray 2024] Arka Ray. "There is no APTAS for 2-dimensional vector bin packing: Revisited". *Information Processing Letters* 183 (Jan. 2024), p. 106430. ISSN: 0020-0190. DOI: 10.1016/j.ipl.2023.106430. URL: http://dx.doi.org/10.1016/j.ipl.2023.106430 (cit. on p. 19).

[Reiss, Tumanov, *et al.* 2012] Charles Reiss, Alexey Tumanov, Gregory R. Ganger, Randy H. Katz, and Michael A. Kozuch. "Heterogeneity and Dynamicity of Clouds at Scale: Google Trace Analysis". In: *Proceedings of the Third ACM Symposium on Cloud Computing.* SoCC '12. San Jose, California: Association for Computing Machinery, 2012. ISBN: 9781450317610. DOI: 10.1145/2391229.2391236. URL: https://doi.org/10.1145/2391229.2391236 (cit. on p. 1).

[Reiss, Wilkes, *et al.* 2012] Charles Reiss, John Wilkes, and Joseph L. Hellerstein. "Obfuscatory obscanturism: making workload traces of commercially-sensitive systems safe to release". In: *3rd International Workshop on Cloud Management (CLOUDMAN).* Maui, HI, USA: IEEE, Apr. 2012, pp. 1279–1286. URL: http://ieeexplore.ieee.org/xpls/abs_all.jsp?arnumber=6212064 (cit. on p. 37).

[Rzadca *et al.* 2020] Krzysztof Rzadca *et al.* "Autopilot: Workload Autoscaling at Google". In: *Proceedings of the Fifteenth European Conference on Computer Systems.* EuroSys '20. Heraklion, Greece: Association for Computing Machinery, 2020. ISBN: 9781450368827. DOI: 10.1145/3342195.3387524. URL: https://doi.org/10.1145/3342195.3387524 (cit. on pp. 2, 11, 13, 17, 19, 29, 55, 56).

[Sheng *et al.* 2023] Junjie Sheng *et al.* "Learning Cooperative Oversubscription for Cloud by Chance-Constrained Multi-Agent Reinforcement Learning". In: *Proceedings of the ACM Web Conference 2023*. WWW '23. Austin, TX, USA: Association for Computing Machinery, 2023, pp. 2927–2936. isbn: 9781450394161. doi: 10.1145/3543507.3583298. url: https://doi.org/10.1145/3543507.3583298 (cit. on p. 56).

[Tchernykh *et al.* 2020] Andrei Tchernykh *et al.* "Toward digital twins' workload allocation on clouds with low-cost microservices streaming interaction". In: *2020 Ivannikov Ispras Open Conference (ISPRAS)*. 2020, pp. 115–121. doi: 10.1109/ISPRAS51486.2020.00024 (cit. on p. 9).

[Verma *et al.* 2015] Abhishek Verma *et al.* "Large-scale cluster management at Google with Borg". In: *Proceedings of the European Conference on Computer Systems (EuroSys)*. Bordeaux, France, 2015 (cit. on p. 37).

[VMware n.d.] VMware. *VMware vSphere Hypervisor (ESXi)*. https://www.vmware.com/products/esxi-and-esx.html. Accessed: 13 March 2025 (cit. on p. 1).

[Wang *et al.* 2023] Yaohui Wang, Ben Luo, and Yibin Shen. "Efficient Memory Overcommitment for I/O Passthrough Enabled VMs via Fine-grained Page Meta-data Management". In: *2023 USENIX Annual Technical Conference (USENIX ATC 23)*. 2023, pp. 769–783 (cit. on p. 17).

[Wilkes 2011] John Wilkes. *More Google cluster data*. Google research blog. Posted at http://googleresearch.blogspot.com/2011/11/more-google-cluster-data.html. Mountain View, CA, USA, Nov. 2011 (cit. on p. 2).

[Wilkes 2020] John Wilkes. *Yet more Google compute cluster trace data*. Google research blog. Posted at https://ai.googleblog.com/2020/04/yet-more-google-compute-cluster-trace.html. Mountain View, CA, USA, Apr. 2020 (cit. on p. 2).

[Xen Project n.d.] Xen Project. *Xen Hypervisor*. https://www.xenproject.org. Accessed: 13 March 2025 (cit. on p. 1).

[Yang *et al.* 2022] Yanan Yang *et al.* "SDCBench: A Benchmark Suite for Workload Colocation and Evaluation in Datacenters". *Intelligent Computing* 2022 (2022). doi: 10.34133/2022/9810691. eprint: https://spj.science.org/doi/pdf/10.34133/2022/9810691. url: https://spj.science.org/doi/abs/10.34133/2022/9810691 (cit. on p. 39).

[W. Zhang *et al.* 2022] Wei Zhang *et al.* "PilotFish: Harvesting Free Cycles of Cloud Gaming with Deep Learning Training". In: *2022 USENIX Annual Technical Conference (USENIX ATC 22)*. 2022, pp. 217–232 (cit. on p. 17).

[Z. Zhang *et al.* 2014] Zhuo Zhang *et al.* "Fuxi: a fault-tolerant resource management and job scheduling system at internet scale". *Proc. VLDB Endow.* 7.13 (Aug. 2014), pp. 1393–1404. issn: 2150-8097. doi: 10.14778/2733004.2733012. url: https://doi.org/10.14778/2733004.2733012 (cit. on p. 17).